

AG1 skripta

Cyril Ovečka, Lukáš Vesecký

9. října 2026

Obsah

1	1. Přednáška	4
1.1	Základy teorie grafů	4
1.2	Algoritmus BFS (Breadth-First Search)	6
1.2.1	Důkazy správnosti BFS	8
1.3	Speciální třídy grafů a vlastnosti	9
1.4	Věta o principu sudosti	12
2	2. Přednáška	15
2.1	Souvislost grafu a souvislé komponenty	15
2.2	Rozklad grafu na souvislé komponenty (BFS_Graf)	16
2.3	Výpočetní model RAM	18
2.4	Reprezentace grafů v paměti počítače	18
2.4.1	Matice sousednosti	19
2.4.2	Seznam sousedů	19
2.4.3	Reprezentace orientovaného grafu	19
2.5	Časová složitost algoritmu BFS	20
2.6	Slabá a silná souvislost orientovaných grafů	21
2.7	Izomorfismus a automorfismus grafů	21
3	3. Přednáška	23
3.1	Stromy	23
3.1.1	Vlastnosti stromu	23
3.2	Ekvivlentní definice stromu	24
3.3	Kostra grafu	27
3.4	Topologické uspořádání	29
3.4.1	Algoritmus TopSort	30
3.5	Problém řazení a kvadratické algoritmy	32
3.5.1	Základní kvadratické algoritmy	32
3.5.2	Algoritmus BubbleSort	33
4	4. Přednáška	34
4.1	Zakořeněné a binární stromy	34
4.2	Binární minimová halda	35
4.2.1	Strukturální vlastnosti haldy	36
4.3	Základní operace nad haldou	37
4.3.1	Vyhledání minima (HeapFindMin)	37
4.3.2	HeapInsert	37
4.3.3	Odebrání minima (HeapExtractMin)	38
4.4	Reprezentace haldy v poli	39
4.5	Rychlá konstrukce haldy (HeapBuild)	39

4.6	Řadící algoritmus HeapSort	40
4.7	Amortizovaná časová složitost	41
4.7.1	Příklad 1: Nafukovací pole	41
4.7.2	Příklad 2: Binární sčítačka	43
5	5. Přednáška	44
5.1	Binární vyhledávací strom	44
5.2	BVS Algoritmy	45
5.2.1	BVSShow	45
5.2.2	BVSMin	45
5.2.3	BVSFind	45
5.2.4	BVSPred	46
5.2.5	BVSInsert	46
5.2.6	BVSDelete	47
5.3	Časová složitost operací nad BVS	48
5.4	Dokonale Vyvážený BVS	48
5.5	Avl strom	49
5.5.1	Důkaz hloubky AVL	49
5.5.2	Rotace AVL	50
5.5.3	Časová složitost operací nad AVL stromem	51

Motivace a úvod

Vítáme vás u studijního textu k předmětu BI-AG1. Tento text je neoficiální materiál pro tento předmět. Pokud byste narazili na jakoukoli chybu, prosím kontaktujte nás ať ji můžeme opravit.

Jak jistě víte, svět se mění (a to nejen v IT). Díky pokročilým LLM již často není programování "jednoduchých" funkcí už úkolem pro vás, ale pro jazykový model. Proto je ale o to důležitější umět navrhovat a pochopit algoritmy, proč a jak fungují a jak jejich korektnost ověřit jinak, než velkým počtem testovacích dat (ty někdy nemusí stačit, také chybu nemusí vůbec odhalit, nebo ji odhalí až potom, co celý algoritmus dokončíte a zjistíte, že musíte začít znovu).

Tento předmět se také věnuje grafům. Je možné, že jste na ně už při studiu narazili, byť možná jen v omezené míře. Ať už v BI-PA2 nebo jako diagram relace v BI-DML. Ale to není zdaleka vše kde se s grafy potkáte. Grafy jsou opravdu všude. Mapa pražského metra? Model počítačové sítě? Hledání cesty v bludišti? Rodokmen? Ano to vše jsou grafy. Tím, že nějaký problém v životě budete reprezentovat grafem a nad ním provádět operace/pouštět algoritmus ho můžete mnohem jednodušeji vyřešit. Například hledání trasy v navigaci je jen hledání nejkratší cesty v grafu. Dají se tím ale řešit i problémy které s grafem zdánlivě nemají ale vůbec nic společného. Například převozníkův problém se dá řešit pomocí reprezentace v grafu a hledáním cesty mezi dvěma vrcholy.

Toto se tedy v tomto kurzu naučíte, pojďme ale k věci a konečně si přesně zdefinovat co to takový graf je.

1. Přednáška

1.1 Základy teorie grafů

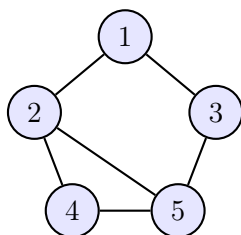
Jak už jsme zmiňovali, budeme se zde zabývat grafy. Proto si nyní první z nich zavedeme.

Definice 1.1 (Neorientovaný graf). Graf G je uspořádaná dvojice (V, E) , kde:

- V je neprázdná konečná množina **vrcholů**,
- E je množina **hran**, přičemž každá hrana je dvouprvková podmnožina V (čili neuspořádaná dvojice vrcholů), značíme $\{u, v\}$.

Množinu všech dvouprvkových podmnožin množiny V (tedy všech možných hran) značíme $\binom{V}{2}$. Obvykle budeme říkat je **graf**, tj. neříkáme neorientovaný.

Dále si všimneme, že hrana je dvouprvková podmnožina vrcholů. Tedy nezáleží na pořadí v jakém vrcholy uvedeme. Zároveň nemůže existovat více hran mezi dvěma vrcholy. Poté by množina hran nebyla množinou. Dále nemůže existovat smyčka, tedy hrana z vrcholu v do toho samého vrcholu. Jelikož by se poté nejednalo o dvouprvkovou podmnožinu vrcholů.



Obrázek 1: Příklad neorientovaného grafu s 5 vrcholy a 6 hranami.

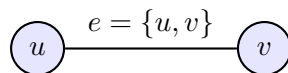
Poznámka 1.2. Množinu vrcholů grafů zavádíme jakožto neprázdnou. Důvodem je aby jsme u většiny důkazů nemuseli explicitně ošetřovat prázdný graf. Poté u datových struktur toto omezení pomineme.

Definice 1.3 (značení). Nechť G je graf. Pak

- Množinu vrcholů grafu G značíme $V(G)$ a množinu hran $E(G)$.
- Počet vrcholů grafu budeme obvykle značit $n = |V(G)|$ a počet hran $m = |E(G)|$.

Definice 1.4 (incidence). Nechť $e = \{u, v\}$ je hrana v grafu G . Pak řekneme, že:

- vrcholy u a v jsou **koncové vrcholy** hrany e ,
- u je **sousedem** v v G (a naopak),
- u i v jsou **incidentní** s hranou e .

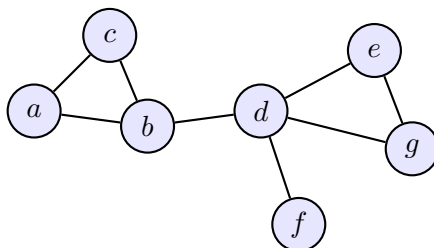


Obrázek 1.2: Vizualizace incidence vrcholu a sousedství přes hranu e .

Nyní si zavedeme pojmy sled a cesta. Sled je vlastně posloupnost vrcholů a hran, které spolu sousedí. Cesta je poté silnější pojem, jelikož nám zakazuje opakování vrcholů (a tedy i hran, laskavý čtenář rozmyslí).

Definice 1.5 (Sled a cesta).

- **Sled** délky k v grafu G je sekvence $v_0, e_1, v_1, e_2, \dots, e_k, v_k$ taková, že $e_i = \{v_{i-1}, v_i\}$ a $e_i \in E(G)$ pro všechna $i = 1, \dots, k$.
- **Cesta** v grafu G je sled, ve kterém se neopakují vrcholy (a tedy ani hrany). Má-li cesta P v grafu G koncové vrcholy $s = v_0$ a $t = v_k$, mluvíme o **cestě z s do t** (nebo o **s - t -cestě**). Připouštíme i $s = t$ (cesta nulové délky).
- **Délka** s - t -cesty je počet hran v této cestě.
- **Vzdálenost** s a t , značná $d(s, t)$, je délka nejkratší s - t -cesty v grafu G .



Obrázek 2: Sekvence a, b, d, e, g, d, f je sled, ale ne cesta (opakuje se vrchol d). Sekvence a, b, d, f je cesta.

1.2 Algoritmus BFS (Breadth-First Search)

Nyní si představíme velmi důležitý algoritmus BFS neboli Breath-First search. Ten slouží pro nalezení nejkratší cesty mezi vrcholy. S jeho znalostí, pokud převedeme náš problém na graf, tak zvládneme najít nejkratší řešení.

Vstup: Neorientovaný graf $G = (V, E)$ a počáteční vrchol $s \in V$.

Výstup:

- Pole vzdáleností D od vrcholu s takové, že:
 - $D[v] = k$, kde k je délka nejkratší s - v -cesty v G ,
 - $D[v] = \text{undef}$, pokud neexistuje žádná s - v -cesta v G .
- Pole předchůdců P takové, že:
 - $P[v] = w$, kde w je předchůdce v na nějaké nejkratší s - v -cestě v G ,
 - $P[v] = \text{undef}$, pokud neexistuje žádná s - v -cesta v G .

Tedy algoritmus má najít délky nejkratších cest mezi počátečním vrcholem s a všemi ostatními. Pokud se ptáte, jak získat samotnou nejkratší cestu (tedy posloupnost vrcholů a hran), tak tu lze jednoduše získat díky poli předchůdců P , stačí se podívat na předchůdce posledního vrcholu a následně jeho předchůdce a toto opakovat dokud se nedostaneme do samotného vrcholu s . Hledaná cesta je pak posloupnost těchto předchůdců.

Jak funguje BFS: Začneme v počátečním vrcholu s . Poté vezmeme všechny jeho ještě nenalezené sousedy a umístíme je do fronty. Na závěr vrchol s uzavřeme a jeho práce končí. Následně opakujeme:

1. Vezmeme první prvek ve frontě, nazvěme ho v .
2. Přidáme hodnotu do pole vzdáleností a pole předchůdců pro vrchol v
3. Přidáme do fronty všechny zatím nenalezené sousedy vrcholu v a nastavíme je jako otevřené
4. Označíme v jako uzavřený

Jakmile již není možné vzít další prvek z fronty (fronta je prázdná) algoritmus končí. Výstupem jsou naplněná pole vzdáleností a předchůdců. Pro ilustraci se můžete podívat do slidů z přednášky, nebo na některé z mnoha videí dostupných na internetu. Pseudokód pak vypadá takto:

BFS(graf G , vrchol s)

- (1) Pro každý vrchol $v \in V(G)$:
- (2) stav[v] := nenalezený
- (3) $D[v] := \text{undef}$
- (4) $P[v] := \text{undef}$
- (5) $Q :=$ fronta obsahující jediný vrchol s
- (6) stav[s] := otevřený
- (7) $D[s] := 0$
- (8) $P[s] := \perp$
- (9) Dokud je fronta Q neprázdná:
- (10) Vyjmi z Q první vrchol, nechť to je v
- (11) Pro všechny sousedy w vrcholu v :
- (12) Pokud stav[w] = nenalezený:

- (13) $\text{stav}[w] := \text{otevřený}$
- (14) $D[w] := D[v] + 1$
- (15) $P[w] := v$
- (16) Přidej w na konec fronty Q
- (17) $\text{stav}[v] := \text{uzavřený}$
- (18) Vrať (D, P)

Nyní co jsme algoritmus zavedli, tak je potřeba ho analyzovat. Dokazuje se většinou několik vlastností. Zejména konečnost, že algoritmus někdy doběhne a správnost tedy že výsledek co dostaneme je správný. Nyní začneme konečností. (U konečnosti se dokazuje i časová složitost, na to zatím nemáme aparát. U BFS doplníme u přednášky 2.)

Věta 1.6 (O konečnosti algoritmu BFS). *Algoritmus $BFS(G, s)$ se vždy zastaví.*

Důkaz. Každý vrchol je přidán do fronty Q nejvýše jednou, neboť před svým vložením je jeho stav změněn na *otevřený* a nemůže již vícekrát splnit podmínku, že je *nenalezený*. V každé iteraci hlavního cyklu je právě jeden vrchol odebrán z fronty. Protože množina vrcholů $V(G)$ je konečná, algoritmus se zastaví nejvýše po $|V(G)|$ iteracích hlavního cyklu. $\square$

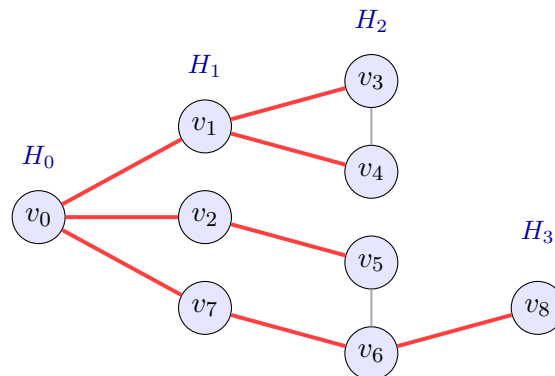
Definice 1.7 (BFS fáze a BFS hladiny).

Hladina $H_0 = \{s\}$. Fáze F_0 trvá od odebrání s z fronty Q , otevření a vložení všech jeho sousedů do Q až do uzavření s .

Hladina H_i (pro $i > 0$) je množina všech vrcholů otevřených a vložených do Q ve fázi F_{i-1} .

Fáze F_i trvá od konce fáze F_{i-1} až po uzavření všech vrcholů z H_i . To zahrnuje vyjmutí všech vrcholů z H_i z fronty Q , otevření jejich dosud nenalezených sousedů a vložení těchto sousedů do Q .

Nechť h je číslo poslední fáze algoritmu BFS.



Obrázek 3: Rozklad grafu na hladiny H_0, H_1, H_2, H_3 podle vzdálenosti od v_0 . Červeně jsou zvýrazněny hrany kostry.

Nyní si dokážeme korektnost algoritmu. Vždy u korektnosti dokazujeme vše co nám algoritmus slibuje. Zde u BFS jsou to 3 věci, že správně uzavře vrcholy, že správně naplní pole vzdáleností a pole předchůdců.

Zde se lišíme od přednášky! V přednášce jsou ty důkazy velmi odměňované, a kdyby byli na zkoušce (jako že tam pravděpodobně nebudou), tak by nemuseli být uznané. Pokud chcete pochopit princip doporučujeme přednášku, zde jsou stejné ale víc formálně popsané. Zde je jediná výjimka kde se lišíme.

Věta 1.8 (O správnosti algoritmu BFS). *Po skončení algoritmu $BFS(G, s)$ platí následující vlastnosti pro každý vrchol $v \in V(G)$:*

- **Vlastnost 1:** *Vrcholy jsou ve stavu uzavřený právě tehdy, když do nich ze startu s vede cesta. Ostatní vrcholy zůstanou nenalezené.*
- **Vlastnost 2:** *Pro všechny uzavřené vrcholy v platí $D[v] = d(s, v)$ (délka nejkratší cesty ze startu s do v).*
- **Vlastnost 3:** *Pro všechny uzavřené vrcholy $v \neq s$ platí $P[v] = w$, kde w je předchůdce v na nějaké nejkratší cestě ze startu s do v .*

1.2.1 Důkazy správnosti BFS

Důkaz Vlastnosti 1. Připomínáme, že prvním otevřeným a následně uzavřeným vrcholem je startovní vrchol s . Algoritmus otevírá všechny dosud nenalezené sousedy právě zpracovávaného vrcholu. Zmíňme ještě, že dosažitelný vrchol říkáme vrcholu, do kterého existuje cesta z vrcholu s (tedy samotný s toto také splňuje).

Jedná se o ekvivalenci, musíme proto ukázat platnost dvou implikací.

- Vrchol v je dosažitelný z $s \implies$ vrchol v bude uzavřen.

Základní krok: Pro cestu délky 0 platí $v = s$. Startovní vrchol s algoritmus otevře na začátku, takže tvrzení platí.

Indukční krok: Předpokládejme, že každý vrchol, ke kterému vede ze s cesta délky k , bude otevřen. Uvažujme nyní vrchol v , ke kterému vede ze s cesta délky $k + 1$. Označme u předposlední vrchol této cesty. Potom ke u vede ze s cesta délky k a mezi u a v existuje hrana.

Podle indukčního předpokladu bude vrchol u otevřen. Při jeho zpracování algoritmus BFS prochází všechny jeho sousedy, tedy i vrchol v . Pokud v ještě nebyl nalezen, BFS jej otevře. Pokud již nalezen byl, byl otevřen dříve. V obou případech tedy bude v během algoritmu otevřen.

- Vrchol v je uzavřený $\implies$ vrchol v je dosažitelný z s
Tvrzení dokážeme indukcí podle pořadí ve kterém BFS vrcholy otevírá.

Základní krok: Prvním otevřeným vrcholem je vrchol s . Vrchol s je dosažitelný z s cestou délky 0.

Indukční krok: Předpokládejme, že vrchol u byl otevřen dříve než vrchol v a že u je dosažitelný z s . Vrchol v je otevřen při zpracování vrcholu u , takže mezi u a v existuje hrana u, v . Nyní můžeme spojit cestu ze s do u s hranou u, v , čímž dostaneme cestu ze s do v . Tedy vrchol v je dosažitelný z s .

□

Myšlenka důkazu. Zvolíme fázi F_i . Z konstrukce BFS víme, že každý vrchol $v \in H_i$ má vzdálenost od s nejvýše i , protože existuje cesta z s do v právě přes i hladin. Zbývá ukázat, že nemůže existovat kratší cesta. Kdyby existovala cesta z s do v kratší než i , musela by při průchodu z H_0 do H_i přeskočit alespoň jednu hladinu. To ale není možné, protože každé dva sousední vrcholy leží buď ve stejné hladině, nebo v hladinách s rozdílem nejvýše 1.

Důkaz Vlastnosti 2. Důkaz provedeme indukcí dle fáze.

- **Základní krok:** Tvrzení triviálně platí pro fázi F_0 , kde $D[s] = 0 = d(s, s)$.
- **Indukční krok:** Uvažujme indukční krok pro fázi F_i kde $i > 0$. Pokud $v \in H_i$, pak podle konstrukce $D[v] = i$ a z s do v existuje cesta o i hranách. Tedy skutečná vzdálenost $d(s, v) \leq i$.

Nyní ukážeme sporem, že nemůže existovat žádná s - v -cesta kratší než i . Předpokládejme, že existuje cesta P z s do v o délce $j < i$. Protože $j < i$, musí podle holubníkového principu na cestě P existovat dva sousední vrcholy w a w' a dvě nesousední hladiny H_k a H_ℓ (tj. $|k - \ell| > 1$) takové, že $w \in H_k$ a $w' \in H_\ell$.

To je ovšem spor s tím, že sousední vrcholy na cestě musí ležet v sousedních hladinách (rozdíl indexů hladin sousedních vrcholů může být nejvýše 1, protože když se zpracovává hladina H_k , všichni její dosud nenalezení sousedé se zařadí do H_{k+1} , a ostatní sousedé již leží v H_k nebo H_{k-1}).

Tento spor dokazuje, že žádná kratší cesta neexistuje, a tedy $d(s, v) = i = D[v]$.

□

Důkaz Vlastnosti 3. Někdy doděláme :). Zatím jsem nepotkal člověka po kterém by byl vyžadován

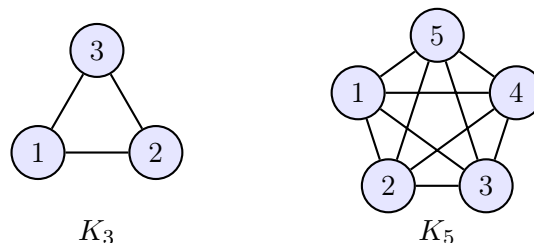
□

Na závěr této části bychom ještě rádi doporučili vyřešit si miniprotest na BFS, nejenže díky tomu BFS lépe pochopíte, ale také vám usnadní řešení protestu velkého.

1.3 Speciální třídy grafů a vlastnosti

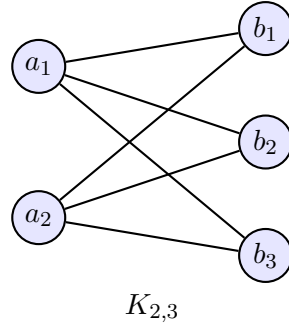
Nyní si zavedeme pár speciálních grafů o kterých se často budeme bavit. Důležité je jejich značení. Úplný graf je K_n , Úplný bipartitní graf je $K_{n,m}$, Kružnice je C_n a Cesta je P_n . Kde n, m značí počet vrcholů.

Definice 1.9 (Úplný graf). Necht' $n \geq 1$. **Úplný graf** na n vrcholech, značný K_n , je graf $(V, \binom{V}{2})$, kde $|V| = n$. V úplném grafu jsou každé dva různé vrcholy spojené hranou. Někdy se úplné grafy také nazývají klika.



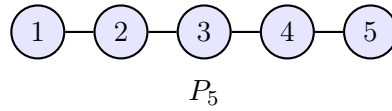
Obrázek 4: Úplné grafy K_3 (3 vrcholy, 3 hrany) a K_5 (5 vrcholů, 10 hran).

Definice 1.10 (Úplný bipartitní graf). Necht' $n_1 \geq 1$ a $n_2 \geq 1$. **Úplný bipartitní graf** K_{n_1, n_2} tvořený dvěma partitami o velikostech n_1 a n_2 je graf $G = (A \cup B, \{\{a, b\} \mid a \in A, b \in B\})$, kde $A \cap B = \emptyset$, $|A| = n_1$, $|B| = n_2$.



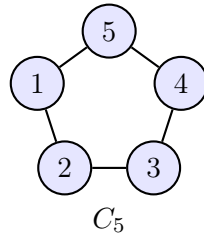
Obrázek 5: Úplný bipartitní graf $K_{2,3}$ s partitami velikosti 2 a 3.

Definice 1.11 (Cesta). Necht' $n \geq 1$. **Cesta** s n vrcholy je graf $P_n = (V, E)$ s množinou vrcholů $V = \{1, \dots, n\}$ a množinou hran $E = \{\{i, i+1\} \mid i \in \{1, \dots, n-1\}\}$.



Obrázek 6: Cesta P_5 s pěti vrcholy.

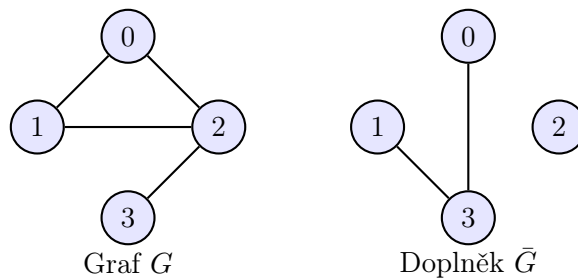
Definice 1.12 (Kružnice). Necht' $n \geq 3$. **Kružnice** délky n je graf $C_n = (V, E)$ s množinou vrcholů $V = \{1, \dots, n\}$ a množinou hran $E = \{\{i, i+1\} \mid i \in \{1, \dots, n-1\}\} \cup \{\{n, 1\}\}$.



Obrázek 7: Kružnice C_5 s pěti vrcholy.

Doplňek grafu je graf, který má úplně stejné vrcholy. A pouze prohodí hrany. Tam kde předtím hrana vedla, tak v doplňku nepovede a ty hrany co v původním grafu nebyly, v doplňku jsou.

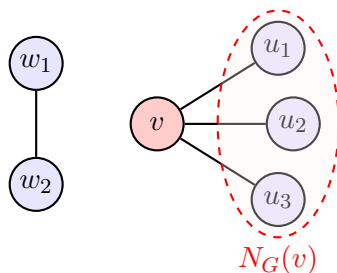
Definice 1.13 (Doplňek grafu). **Doplňek** $\bar{G}$ grafu $G = (V, E)$ je graf $(V, \binom{V}{2} \setminus E)$.



Obrázek 8: Příklad grafu G a jeho doplňku $\bar{G}$ na čtyřprvkové množině vrcholů.

Definice 1.14 (Okolí a stupně). Nechť $G = (V, E)$ je graf a $v \in V$ jeho vrchol.

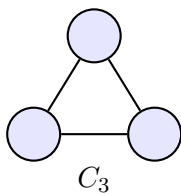
- Symbolem $\deg_G(v)$ označíme počet hran grafu G obsahujících vrchol v .
- Symbolem $N_G(v)$ označíme **otevřené okolí** vrcholu v , tj. množinu všech sousedů v . Platí $\deg_G(v) = |N_G(v)|$.
- Množinu $N_G[v] = N_G(v) \cup \{v\}$ nazveme **uzavřeným okolím** vrcholu v .



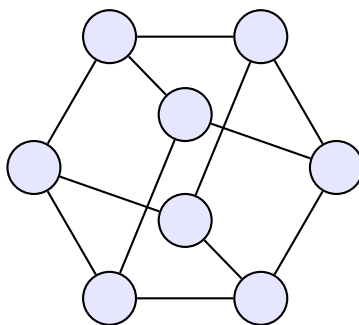
Obrázek 9: Okolí $N_G(v)$ vrcholu v . Vrcholy w_1 and w_2 nepatří do okolí v .

Definice 1.15 (Regulární graf).

- Graf je **r -regulární**, pokud stupeň každého jeho vrcholu je r .
- Graf je **regulární**, pokud je r -regulární pro nějaké r .



Obrázek 10: Příklad 2-regulárního grafu C_3 .



Obrázek 11: Příklad 3-regulárního grafu na 8 vrcholech (C_6 se dvěma vnitřními vrcholy).

Definice 1.16 (Izolovaný vrchol). **Izolovaný vrchol** je vrchol stupně 0, tedy vrchol, který nemá žádné sousedy.

1.4 Věta o principu sudosti

Princip sudosti nám říká, že každý graf musí mít dohromady sudý počet stupňů. Proč je to pravda? Každá hrana má dva vrcholy a každý z těchto vrcholů si započítá 1 stupeň. Tedy za každou hranu máme 2 stupně.

Věta 1.17 (O principu sudosti / Handshaking lemma). *Pro každý graf $G = (V, E)$ platí:*

$$\sum_{v \in V} \deg_G(v) = 2|E|$$

Důkaz. Při sečtení stupňů všech vrcholů započítáme každý konec každé hrany právě jednou. Jelikož každá hrana má přesně dva konce, započítáme každou hranu přesně dvakrát.

Formálně to můžeme vyjádřit následujícím řetězcem rovností:

$$\sum_{v \in V} \deg_G(v) = \sum_{v \in V} |N_G(v)| = \sum_{v \in V} |\{u \in V \mid \{u, v\} \in E\}|$$

Tedy součet stupňů všech vrcholů je roven počtu jeho sousedů (přímo definice).

To vyjádříme dvěma sumami:

$$\sum_{v \in V} \sum_{\substack{e \in E \\ v \in e}} 1$$

Vnější suma je přes všechny vrcholy $v \in V(G)$ a vnitřní suma sčítá přes všechny hrany současného vrcholu v .

Prohozením pořadí sumace (přesčítáním přes všechny hrany a jejich konce, tedy vždy 2 vrcholy):

$$\sum_{e \in E} \sum_{v \in e} 1 = \sum_{e \in E} 2 = 2|E|$$

Dostáváme požadovaný počet. □

Důsledek 1.18. *Z věty o principu sudosti plynou tyto důležité vlastnosti:*

1. *V každém grafu je počet vrcholů lichého stupně sudý.*
2. *Každý regulární graf lichého stupně musí mít sudý počet vrcholů.*

Princip sudosti nám dává nutnou podmínku která se, jak už to jistě znáte, skvěle hodí pro vyvracení existence grafů. Pokud byste například dostali za úkol sestavit graf tak, aby obsahoval vrcholy se zadanými stupni, pokud součet stupňů bude lichý hned víte, že to nelze (nemusíte sčítat samotné stupně, stačí se jen podívat na počet vrcholů s lichým stupněm a pokud jich nebude sudě tak víte, že součet stupňů musí být taky lichý). Také se tato věta velmi hodí u důkazů sporem, kde často můžete nalézt spor právě s touto větou.

Teď si zavedeme podgrafy. Podgraf je podmnožina původního grafu, ale pouze tak že je stále grafem. **Důležité:** prázdný graf není grafem a celý graf je podgrafem sama sebe.

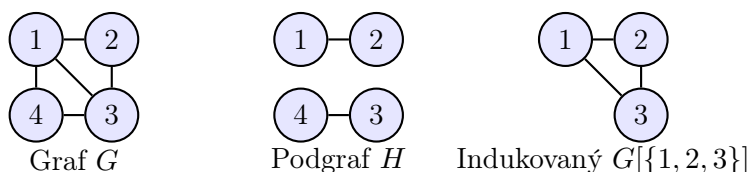
Více omezení jsme u indukovaného podgrafu, který po vybrání podmnožiny vrcholů musí zahrnout všechny hrany co mezi libovolnou dvojicí z nich vedli.

Definice 1.19 (Podgraf a indukovaný podgraf).

- Graf H je **podgrafem** grafu G (píšeme $H \subseteq G$), pokud $V(H) \subseteq V(G)$ a $E(H) \subseteq E(G)$.
- Graf H je **indukovaným podgrafem** grafu G , pokud $V(H) \subseteq V(G)$ a množina hran $E(H)$ obsahuje právě ty hrany z G , které mají oba koncové vrcholy ve $V(H)$, tedy:

$$E(H) = E(G) \cap \binom{V(H)}{2}$$

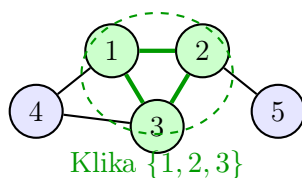
- Indukovaný podgraf na množině vrcholů $V' \subseteq V$ značíme $G[V']$.
- Graf $G - V'$ značí podgraf indukovaný množinou $V \setminus V'$. Pro jednoprvkovou množinu píšeme zkráceně $G - v$.



Obrázek 12: Příklad grafu G , jeho podgrafu H (chybí některé hrany) a indukovaného podgrafu na množině vrcholů $\{1, 2, 3\}$ (obsahuje všechny dostupné vnitřní hrany).

Definice 1.20. Podgraf úplného bipartitního grafu se nazývá **bipartitní graf**.

Definice 1.21 (Klika). **Klika** v grafu G je podmnožina vrcholů $C \subseteq V(G)$, v níž jsou každé dva vrcholy spojené hranou.



Obrázek 13: Graf obsahující kliku velikosti 3 (zvýrazněná zeleně).

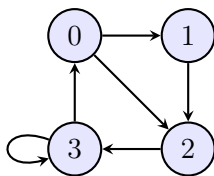
Na závěr si zavedeme druhý typ grafu a to orientovaný graf. V orientovaném grafu řešíme směr, vrcholu z říkáme předchůdce a vrcholu po říkáme následník. A navíc nám oproti neorientovanému dovoluje smyčky. Tedy hranu na jednom vrcholu. Tyto grafy stále splňují vše co jsme si zavedli.

Definice 1.22 (Orientovaný graf). **Orientovaný graf** G is uspořádaná dvojice (V, E) , kde:

- V is neprázdňá konečná množina **vrcholů**,
- E is množina **orientovaných hran**. Orientovaná hrana $e = (u, v)$ is uspořádaná dvojice vrcholů $u, v \in V$ (zobrazujeme jako šipku z u do v).

Říkáme, že u je **předchůdce** v a v je **následník** u .

Orientovaná hrana tvaru (u, u) se nazývá **smyčka**.



Obrázek 14: Příklad orientovaného grafu se smyčkou na vrcholu 3.

BFS lze poměrně lehce modifikovat, tak že přijímá i orientované grafy. Stačí nahradit sousedy za následníky, jelikož neuvažujeme sousedy ze kterých jde šípka k nám.

BFS-Orientovaný(orientovaný graf G , vrchol s)

- (1) Pro každý vrchol $v \in V(G)$:
- (2) stav[v] := nenalezený
- (3) $D[v]$:= undef
- (4) $P[v]$:= undef
- (5) Q := fronta obsahující jediný vrchol s
- (6) stav[s] := otevřený
- (7) $D[s]$:= 0
- (8) $P[s]$:= $\perp$
- (9) Dokud je fronta Q neprázdná:
- (10) Vyjmi z Q první vrchol, nechť to je v
- (11) Pro všechny následníky w vrcholu v :
- (12) Pokud stav[w] = nenalezený:
- (13) stav[w] := otevřený
- (14) $D[w]$:= $D[v] + 1$
- (15) $P[w]$:= v
- (16) Přidej w na konec fronty Q
- (17) stav[v] := uzavřený
- (18) Vrať (D, P)

2 2. Přednáška

2.1 Souvislost grafu a souvislé komponenty

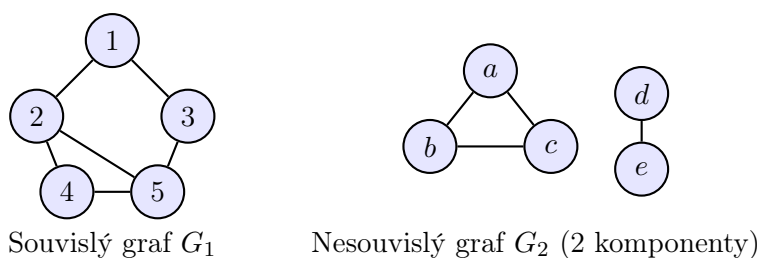
Souvislost je jednou ze základních vlastností grafů. Díky ní víme, že z libovolného vrcholu v se dostaneme do libovolného vrcholu u .

Definice 2.1 (Souvislý graf). Neorientovaný graf G je **souvislý**, jestliže v něm pro každé jeho dva vrcholy $u, v \in V(G)$ existuje u - v -cesta. Jinak je graf G **nesouvislý**.

V případě, že je graf G nesouvislý. Nás ale mohou zajímat jeho podgrafy, které již jsou souvislé. Ty se nazývají souvislé komponenty grafu G .

Definice 2.2 (Souvislá komponenta). Indukovaný podgraf H grafu G je **souvislou komponentou**, pokud je souvislý a neexistuje žádný souvislý podgraf F grafu G takový, že $H \subseteq F$ a $H \neq F$.

Souvislé komponenty jsou tedy největší možné podgrafy grafu G takové, že jsou souvislé.



Obrázek 15: Ukázka souvislého grafu G_1 a nesouvislého grafu G_2 s dvěma komponentami indukovanými množinami $\{a, b, c\}$ a $\{d, e\}$.

Pozorování 2.3. Graf je souvislý právě tehdy, když obsahuje jedinou souvislou komponentu.

Pozorování 2.4. Graf na n vrcholech a 0 hranách má n souvislých komponent.

Souvislost lze také popsat pomocí relace ekvivalence na vrcholech grafu.

Věta 2.5. Binární relace $\leftrightarrow$ nad vrcholy grafu $G = (V, E)$ definovaná předpisem:

$$u \leftrightarrow v \iff \text{existuje } u\text{-}v\text{-cesta v } G$$

je relací ekvivalence. Třídy ekvivalence této relace indukují souvislé komponenty grafu G .

Předtím, než se podíváte na důkaz proč je tato relace ekvivalencí, zkuste si to jako cvičení dokázat sami, u prvních dvou vlastností se jedná se o velmi přímočarý důkaz a velmi dobré cvičení na ověření si, zda jste tuto definici správně pochopili. U třetí vlastnosti si pak musíte uvědomit, že složením dvou cest nemusí vzniknout cesta, ale sled (ten pak lze zkrátit na cestu, na tento důkaz narazíme v druhé půlce semestru).

Důkaz. Stačí nám ověřit reflexivitu, symetrii a transitivitu (viz. DML), tím dokážeme ekvivalenci.

Mějme graf G a nějaké jeho 3 vrcholy $u, v, w \in G$.

1. Reflexivita - Z vrcholu v určitě existuje cesta do vrcholu v . Je to přesně cesta délky 0.
2. Symetrie - Pokud existuje u - v -cesta, tak využitím stejných hran a vnitřních vrcholů. Pouze nyní směrem z vrcholu u do vrcholu v . Získáme hledanou v - u -cestu.
3. Transitivita - Pokud existuje u - v -cesta a v - w -cesta, tak poté spojením těchto dvou cest nám vznikne sled z u do w . Ten lze ale zkrátit na cestu (to je netriviální, důkaz v 10. přednášce).

□

2.2 Rozklad grafu na souvislé komponenty (BFS_Graf)

Pokud graf tedy není souvislý, nemůžeme jednoduše pustit algoritmus BFS na libovolném vrcholu a očekávat, že nalezne všechny vrcholy, jak to tedy zařídit? Zařídí to tato mírně modifikovaná verze algoritmu.

Algoritmus **BFS_Graf** prochází celý graf G a postupně spouští algoritmus BFS pro každý dosud nenalezený vrchol $s \in V(G)$. Tímto způsobem postupně projde všechny souvislé komponenty grafu a spočítá vzdálenosti D i strom nejkratších cest P pro každou komponentu zvlášť. Nemůže tak zůstat vrchol do , kterého by se algoritmus nedostal, jelikož bude provolán vnější smyčkou a sám najde všechny ze svojí komponenty.

Poznámka 2.6. Toto není jediný způsob prohledání grafu, algoritmus BFS lze v tomto případě například zaměnit za DFS.

Vstup: Neorientovaný graf $G = (V, E)$.

Výstup:

- Pole vzdáleností D pro všechny vrcholy $v \in V(G)$ od startovních vrcholů jejich příslušných souvislých komponent.
- Pole předchůdců P určující stromy nejkratších cest v jednotlivých komponentách.

BFS_Graf(graf G)

- (1) Pro každý vrchol $u \in V(G)$:
- (2) stav[u] := nenalezený
- (3) $D[u]$:= undef
- (4) $P[u]$:= undef
- (5) Pro každý vrchol $s \in V(G)$:
- (6) Pokud stav[s] = nenalezený:
- (7) Zavolej **BFS**(G, s)
- (8) Vrať (D, P)

Důkaz Korektnosti algoritmu BFS_Graf. Dokážeme přímo

1. U algoritmus BFS jsme již dokázali, že správně vyplní pole předchůdců, pole vzdáleností a nevynechá žádný vrchol.

2. Algoritmus BFS_Graf nejprve nastaví všechny vrcholy na stav *nenalezený*. Poté postupně prochází všechny vrcholy $s \in V(G)$. Pokud je vrchol s stále *nenalezený*, zavolá na něm algoritmus BFS.
3. Uvažujme libovolnou komponentu souvislosti C grafu G . Pokud ještě žádný vrchol z C nebyl nalezen, algoritmus při průchodu vrcholů narazí na některý vrchol $s \in C$ ve stavu *nenalezený* a zavolá BFS(G, s). Protože všechny vrcholy v C jsou z s dosažitelné, algoritmus BFS nalezne všechny vrcholy komponenty C .
4. Po dokončení tohoto volání jsou všechny vrcholy komponenty C označeny jako uzavřené. Když tedy algoritmus později narazí na další vrchol z této komponenty, podmínka stav[s] = *nenalezený* již nebude splněna a další volání BFS nebude provedeno.
5. Každá komponenta souvislosti je tedy zpracována právě jedním voláním algoritmu BFS a žádná komponenta není vynechána. Jelikož je BFS korektní, jsou pro všechny vrcholy správně určeny hodnoty D a P .
6. Algoritmus BFS_Graf je proto korektní.

□

Teď tedy víme, jak prohledat celý graf i pokud předem nevíme, zda je souvislý. Můžete se ale ptát, jak ověřit souvislost grafu? Jak najít jeho souvislé komponenty? Pokud dostanu zadaný vrchol jak najít jeho komponentu souvislosti? Na tyto otázky nyní odpovíme malým "vylepšením" algoritmu, který jsme si právě ukázali.

Dostaneme-li vrchol s jak najít jeho souvislou komponentu? K tomuto nám stačí algoritmus který jsme si právě ukázali, stačí jen přidat jedno pole identifikátorů které pro každý vrchol držet identifikátor komponenty souvislosti ve které se vrchol nachází, to algoritmus vyplní vrcholem ve kterém začalo prohledávání současné komponenty.

2.3 Výpočetní model RAM

Z PAx jistě znáte časovou a paměťovou složitost programu. Abychom ale mohli analyzovat složitosti algoritmu, musíme si zadefinovat nějaký model počítače. Počítače se často hodně liší, různé instrukce trvají jinak dlouho, atd. Proto si zavedeme teoretický model počítače, který modeluje to, co všichni přibližně od běžného počítače očekáváme.

Definice 2.7 (Model RAM). Výpočetní model **RAM** se skládá ze dvou hlavních částí:

- **Paměť:** Je tvořena adresovatelným polem celočíselných paměťových buněk. Adresy jsou celá čísla.
- **Program:** Je konečná posloupnost sekvenčně prováděných instrukcí. Instrukce dělíme na:
 1. **Aritmeticko-logické:** (sčítání, násobení, porovnávání, ...): Obvykle mají dva vstupní argumenty a jeden výstupní. Vstupem může být přímá konstanta, přímo adresovaná buňka, nebo nepřímo adresovaná buňka (adresa je uložena v jiné přímo adresované buňce).
 2. **Řídící:** Zahrnují nepodmíněné skoky, podmíněné skoky a instrukci pro zastavení programu (halt).

Na začátku výpočtu jsou vstupní data uložena v určených paměťových buňkách a procesor v každém kroku provede právě jednu instrukci. Po zastavení se obsah určených buněk interpretuje jako výstup.

Poznámka 2.8. Pro představu může sloužit např. Assembler z předmětu SAP nebo APS. ADD je příkladem aritmeticko-logické instrukce a JMP příkladem řídící instrukce. Na vstup dostanou buďto konstantu, registr, nebo hodnotu z paměti.

Nyní se konečně dostáváme k tomu co je časová a paměťová složitost. Je to ten nejhorší případ, ze všech vstupů až do velikosti n včetně.

Definice 2.9 (Složitost programu v modelu RAM). Složitost algoritmu v nejhorším případě pro vstup velikosti n definujeme následovně:

- **Velikost vstupu** n je počet paměťových buněk, které vstup zabírá.
- **Časová složitost** je maximum z počtu vykonaných instrukcí přes všechny povolené vstupy velikosti nejvýše n .
- **Paměťová (prostorová) složitost** je maximum z počtu použitých paměťových buněk přes všechny povolené vstupy velikosti nejvýše n .

Co nás bude u algoritmů zajímat je, jak budou složitosti programu škálovat s velikostí vstupu, tedy nebudeme počítat přesný počet instrukcí, ale spíše zda algoritmus vykoná za jeho běh $\mathcal{O}(n)$ instrukcí nebo třeba $\mathcal{O}(n^2)$ či dokonce $\mathcal{O}(2^n)$.

2.4 Reprezentace grafů v paměti počítače

Samotný graf můžeme v modelu RAM reprezentovat různými způsoby. Některé způsoby jsou lepší na rychlejší vyhledávání, jiné třeba víc spoří paměť. Při programování zvolte způsob dle vašich požadavků. Zde si představíme dva možné a to matici sousednosti a seznam sousedů.

2.4.1 Matice sousednosti

Definice 2.10 (Matice sousednosti). Necht $G = (V, E)$ je neorientovaný graf s množinou vrcholů $V = \{v_1, v_2, \dots, v_n\}$. **Matice sousednosti** grafu G je čtvercová matice $A_G = (a_{i,j})_{i,j=1}^n$ typu $n \times n$ definovaná předpisem:

$$a_{i,j} = \begin{cases} 1 & \text{pokud } \{v_i, v_j\} \in E, \\ 0 & \text{jinak.} \end{cases}$$

Poznámka 2.11. Pro neorientovaný graf je matice A_G vždy symetrická podle hlavní diagonály. Diagonální prvky $a_{i,i}$ jsou nulové (neorientovaný graf nemá smyčky). Paměťová složitost této reprezentace je vždy $\mathcal{O}(n^2)$, což je nevýhodné pro řídké grafy ($m \ll n^2$). Výhodou je zarovnaný přístup do paměti a díky tomu i dobrá cache.

2.4.2 Seznam sousedů

Definice 2.12 (Seznam sousedů). V reprezentaci **seznamem sousedů** uchováváme pro každý vrchol $v \in V(G)$ jednosměrně či obousměrně zřetězený seznam všech jeho sousedů (tedy vrcholů w , pro které $\{v, w\} \in E$).

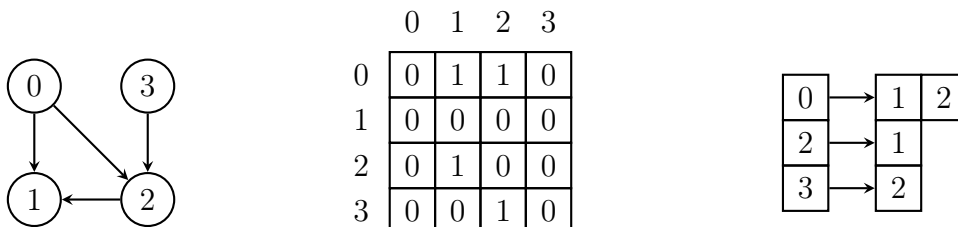
Poznámka 2.13. Paměťová složitost se skládá z pole o velikosti $|V| = n$ pro začátky seznamů a z celkem $2|E| = 2m$ prvků v seznamech (každá neorientovaná hrana je uložena dvakrát – u obou svých koncových vrcholů). Celková paměťová složitost je tedy $\mathcal{O}(n + m)$, což je optimální a velmi efektivní pro řídké grafy. V progtestech jsem volil tento způsob a prošel mi naprosto v pohodě:D

2.4.3 Reprezentace orientovaného grafu

U orientovaných grafů reprezentujeme hrany jako uspořádané dvojice:

- **Matice sousednosti** obsahuje $a_{i,j} = 1$ právě tehdy, když v grafu existuje orientovaná hrana $(v_i, v_j) \in E$. Tato matice obecně **není symetrická**.
- **Seznam sousedů** nahrazujeme **seznamem následníků**: pro každý vrchol v ukládáme pouze seznam vrcholů w takových, že do nich vede orientovaná hrana $(v, w) \in E$. Celkový počet prvků v seznamech je právě $|E| = m$.

$$G = (\{0, 1, 2, 3\}, \{(0, 1), (0, 2), (2, 1), (3, 2)\})$$



(a) Orientovaný graf G (b) Matice sousednosti A_G (c) Seznam následníků

Obrázek 16: Orientovaný graf G a jeho reprezentace maticí sousednosti a seznamem následníků.

2.5 Časová složitost algoritmu BFS

Díky zavedení modelu RAM nyní můžeme konečně dokázat časovou složitost algoritmu BFS. Na základě definovaného modelu RAM a reprezentace grafu můžeme exaktně dokázat časovou složitost algoritmu BFS.

Věta 2.14 (O časové složitosti algoritmu BFS). *Algoritmus $BFS(G, s)$ má při reprezentaci grafu G seznamem sousedů časovou složitost $\mathcal{O}(|V| + |E|)$.*

Důkaz. Časovou složitost můžeme rozdělit na analýzu jednotlivých fází algoritmu:

1. **Inicializace** (řádky 1–3 a 5–6 v neorientované verzi):

- Procházíme všechny vrcholy a na každém provedeme konstantě mnoho práce. Nastavujeme jejich stav na *nenalezený*, vzdálenost na *undef* a předchůdce na *undef*. Celkem to stihneme za $\mathcal{O}(|V|)$.

2. **Práce s frontou Q** (hlavní cyklus na řádcích 7–15):

- Každý vrchol $v \in V(G)$ je do fronty Q přidán nejvýše jednou, protože se jeho stav bezprostředně před vložením změní na *otevřený*. Vrchol v tomto stavu již nemůže splnit podmínku na řádku 10 a do fronty se podruhé nedostane.
- V každé iteraci hlavního cyklu je právě jeden vrchol z fronty odebrán. Hlavní cyklus se tedy provede nejvýše $|V|$ -krát. Každá operace s frontou zabere konstantní množství času. Tedy celkem $\mathcal{O}(|V|)$.

3. **Prohledávání sousedů** (vnitřní cyklus na řádcích 9–14):

- Když je vrchol v vyjmut z fronty, projdeme celý jeho seznam sousedů. Což je nejhůře až $\mathcal{O}(|E|)$.
- V neorientovaném grafu se na každou hranu podíváme nejvýše dvakrát (jednou z každého jejího koncového vrcholu). Celkový počet zpracování hran je tedy nejvýše $2|E|$, což je $\mathcal{O}(|E|)$.

Celkem tedy dostáváme $\mathcal{O}(|V| + |E|)$. □

Je důležité opravdu psát $\mathcal{O}(|V| + |E|)$ a ne jen $\mathcal{O}(|V|)$ nebo $\mathcal{O}(|E|)$, protože nevíme, zda je v grafu více vrcholů nebo hran. Také je dobré si uvědomit, že počet hran v grafu může být až kvadrát počtu vrcholů.

Věta 2.15 (O časové složitosti algoritmu BFS_Graf). *Algoritmus $BFS_graf(G)$, má při reprezentaci grafu seznamem sousedů celkovou časovou složitost $\mathcal{O}(|V| + |E|)$.*

Důkaz. Algoritmus vnějším cyklem projde všechny vrcholy $v \in V(G)$ postupně. Zvolme si libovolnou komponentu souvislosti C grafu G . V jednom z vrcholů začneme BFS a projdeme jí za $\mathcal{O}(|V(C)| + |E(C)|)$. Jelikož komponenty nemají žádnou společnou část. Tedy průchodem nic neopakujeme. Tak projdeme celý graf za sumu přes všechny komponenty. Což je právě celý graf. Tedy $\mathcal{O}(|V| + |E|)$. □

2.6 Slabá a silná souvislost orientovaných grafů

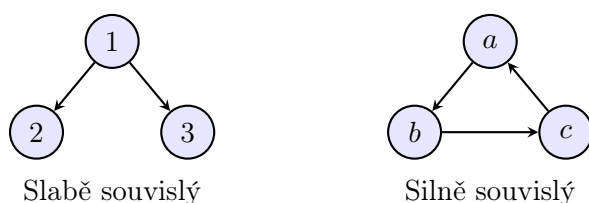
Pro orientované grafy je potřeba pojem souvislosti trochu rozšířit. Rozlišujeme dva druhy. Slabou a silnou souvislost. U Slabé souvislosti se ke hranám chováme, jako by byly neorientované. U Silné souvislosti naopak požadujeme, aby existovala orientovaná cesta mezi libovolnými dvěma vrcholy. Prvně si zavedeme pomocnou definici a to symetrizaci grafu neboli to již zmiňované ignorování orientace hran.

Definice 2.16 (Symetrizace orientovaného grafu). Nechť $G = (V, E)$ je orientovaný graf. **Symetrizace** grafu G je neorientovaný graf $\text{sym}(G) = (V, E')$, kde pro každé dva vrcholy $u, v \in V$ platí:

$$\{u, v\} \in E' \iff (u, v) \in E \vee (v, u) \in E$$

Definice 2.17 (Slabá souvislost). Orientovaný graf G je **slabě souvislý**, pokud je souvislá jeho symetrizace $\text{sym}(G)$.

Definice 2.18 (Silná souvislost). Orientovaný graf $G = (V, E)$ je **silně souvislý**, pokud pro každé dva vrcholy $u, v \in V$ existuje v G orientovaná cesta z u do v a zároveň orientovaná cesta z v do u .



Obrázek 17: Ukázka slabě souvislého grafu (z vrcholu 2 se nelze dostat do ostatních vrcholů, ale po provedení symetrizace je souvislý) a silně souvislého grafu (obsahuje orientovaný cyklus).

2.7 Izomorfismus a automorfismus grafů

Často se může stát, kdy se bavíme o dvou grafech, které jsou strukturou identické, ale jsou například jinak nakreslené nebo pokud mají jinak označené vrcholy. Abychom vyjádřili do jisté míry rovnost těchto grafů, zavádíme následující pojmy.

Definice 2.19 (Izomorfismus grafů). Nechť G a H jsou dva grafy. Zobrazení $f : V(G) \rightarrow V(H)$ nazveme **izomorfismem grafů** G a H , pokud platí:

1. f je bijekce
2. pro každou dvojici vrcholů $u, v \in V(G)$ platí:

$$\{u, v\} \in E(G) \iff \{f(u), f(v)\} \in E(H)$$

Pokud takové zobrazení existuje, říkáme, že grafy G a H jsou **izomorfní** and značíme $G \simeq H$.

Definice 2.20 (Automorfismus grafu). **Automorfismus** grafu G je izomorfismus grafu G se sebou samým, tedy bijekce $f : V(G) \rightarrow V(G)$ zachovávající relaci sousedství.

Poznámka 2.21 (Počty grafů). Na pevné n -prvkové množině vrcholů V existuje právě $2^{\binom{n}{2}}$ různých grafů. Počet navzájem neizomorfních grafů je však výrazně nižší. Například pro $n = 3$ existuje $2^3 = 8$ různých grafů, které se však dělí do pouhých 4 tříd izomorfismu. Ty je zrovna potřeba napočítat manuálně. Binomiiální číslo $\binom{n}{k}$ udává počet podmnožin velikosti k z dané množiny velikosti n .

3 3. Přednáška

3.1 Stromy

Nyní se budeme věnovat třídě grafů nazývanou stromy. Je to jedna z nejdůležitějších skupin. Budeme je využívat v algoritmech stejně tak jako v mnoha datových strukturách. Pojdme si formálně zadefinovat stromy a podívat se na jejich vlastnosti.

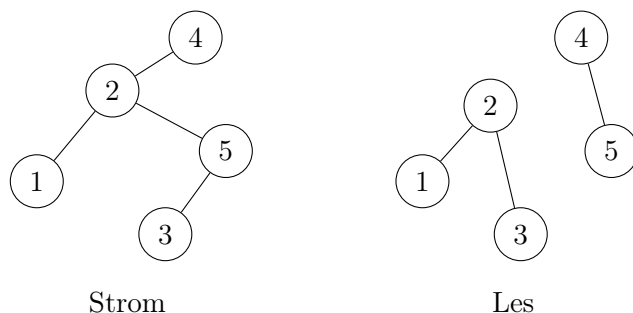
Definice 3.1. Graf G je **strom**, právě tehdy když je souvislý a acyklický.

Strom je tedy libovolný souvislý graf, který neobsahuje cykly. Pro nesouvislý acyklický graf používáme název **les**.

Definice 3.2. Graf G je **les**, právě když je acyklický.

Definice 3.3. Vrchol v v grafu G nazveme **list**, pokud platí $\deg_G(v) = 1$.

List je tedy jakýkoli vrchol v libovolném grafu stupně 1. Ještě si dovolíme poznámku: V anglických textech se list (tedy *leaf*) používá pouze pro vrcholy stupně 1 v lesích nebo stromech. V obecném grafu se nazývají *pendent vertex*.



Obrázek 18: Obrázek stromu a lesa. Oba grafy mají 4 listy.

3.1.1 Vlastnosti stromu

Věta 3.4 (Věta o existenci listů). *Každý strom T s alespoň 2 vrcholy obsahuje alespoň dva listy.*

Myšlenka důkazu: Vybereme si nějakou nejdelší cestu ve stromě a tvrdíme, že její krajní vrcholy jsou listy. Pokud si představíte, jak taková cesta musí vypadat, mělo by být vidět, jak jsme k tomuto tvrzení dospěli. Spor získáme jednoduše tak, že tuto nejdelší cestu prodloužíme. Prodlužování nejdelších cest je velmi časté v důkazech sporem a jistě se vám bude ještě hodit.

Důkaz. Pro spor nechť je graf G strom a P je nějaká nejdelší cesta v grafu G s krajními vrcholy u a v . Předpokládejme, že alespoň jeden z nich není list v grafu G .

Ať je u krajním vrcholem, který není list. Musí tedy mít alespoň 2 hrany (nemůže mít 0, protože je součástí cesty). Označme $e = \{u, w\}$ hranu, která není obsažena v P . Nyní rozlišujeme dva případy:

1. Pokud by byl vrchol w součástí cesty P , pak by v grafu byl cyklus. To je spor s tím, že G je strom (strom je acyklický).

2. Pokud by w součástí cesty nebyl, pak lze cestu P prodloužit přidáním vrcholu w za vrchol u . To je spor s předpokladem, že P je nejdelší cesta v G .

V obou případech jsme dospěli ke sporu. Tvrzení tedy platí. $\square$

Další důležitou vlastností stromů je ta, že pokud odebereme nebo přidáme do grafu list, graf stále zůstane stromem.

Pro jednoduchost zavedeme tuto notaci $G - v = (V \setminus \{v\}, E \setminus \{e \in E | v \in e\})$. Tedy $G - v$ je jednoduše Graf G bez vrcholu v . Podobně pak i notaci $G + v$.

Věta 3.5 (Věta o trhání listů). *Nechť $G = (V, E)$ je graf s alespoň 2 vrcholy a nechť $v \in V$ je jeho list. Pak jsou následující tvrzení ekvivalentní:*

1. G je strom,
2. $G - v$ je strom.

Důkaz. Dokážeme ekvivalenci obou tvrzení pomocí dvou implikací:

$1 \Rightarrow 2$ (Předpokládáme, že G je strom, a dokazujeme, že $G - v$ je strom, tj. potřebujeme dokázat souvislost a acykličnost):

- **$G - v$ je souvislý:** Zvolme libovolné dva vrcholy $x, y \in V \setminus \{v\}$ a cestu P mezi nimi v G (ta existuje díky souvislosti G). Všechny vrcholy cesty P kromě krajních mají stupeň 2, tedy vrchol v nemůže být součástí této cesty, to znamená že mezi všemi dvojicemi vrcholů stále existuje tato cesta i po odebrání v .
- **$G - v$ je bez kružnic:** Odebráním vrcholu z acyklického grafu jsme jistě nemohli vytvořit cyklus.

Graf $G - v$ je souvislý a bez kružnic, je to tedy strom.

$2 \Rightarrow 1$ (Předpokládáme, že $G - v$ je strom, a dokazujeme, že G je strom):

- **G je bez kružnic:** Přidáním listu v ke stromu $G - v$ (vrácením listu v do grafu G) nelze vytvořit cyklus. Jelikož $\deg(v) = 1$.
- **G je souvislý:** Mezi každými dvěma vrcholy $x, y \in V \setminus \{v\}$ vede cesta již v $G - v$. Ať je jediný soused listu v vrchol w . Mezi ním a všemi ostatními vrcholy grafu $G - v$ existuje cesta. Z vrcholu v do libovolného vrcholu u grafu G vznikne cesta tak, že pouze prodloužíme $u-w$ cestu o hranu $\{w, v\}$.

Graf G je tedy souvislý a bez kružnic, což znamená, že je to strom. $\square$

3.2 Ekvivalentní definice stromu

Každý pojem jde zadefinovat více způsoby a v určitých případech se nám i hodí alternativní definice. Proto si zde ukážeme alternativní definice ke stromu. Jediné co je potřeba je ukázat ekvivalenci těchto definic.

Věta 3.6 (Věta o charakterizaci stromů). *Nechť $G = (V, E)$ je graf. Pak jsou následující tvrzení ekvivalentní:*

1. G je strom,

2. pro každé dva vrcholy $u, v \in V$ existuje právě jedna u - v -cesta,
3. G je souvislý a vynecháním libovolné hrany vznikne nesouvislý graf,
4. G je souvislý a $|E| = |V| - 1$.

Poznámka 3.7. Důkazy tvrzení jsou vcelku přímočaré a alespoň před 2. zápočtovou písemkou byste je měli být schopni vymyslet sami. Doporučuji se tedy před ní vrátit a tyto důkazy si zkusit.

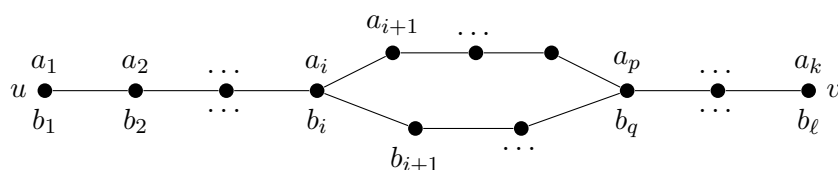
Důkaz. Důkaz ekvivalence provedeme postupným dokázáním jednotlivých implikací:

Dokážeme $1 \Rightarrow 2$: Předpokládejme, že G je strom. Jelikož je G souvislý, existuje pro každé $u, v \in V$ alespoň jedna u - v -cesta.

Pro spor předpokládejme, že existují dva vrcholy u, v , mezi kterými vedou alespoň dvě různé cesty $P_1 = a_1, a_2, \dots, a_k$ a $P_2 = b_1, b_2, \dots, b_\ell$ (kde $a_1 = b_1 = u$ a $a_k = b_\ell = v$).

Nechť $i \geq 1$ je nejmenší index, na kterém se cesty rozcházejí (tj. $a_i = b_i$, ale $a_{i+1} \neq b_{i+1}$). Nechť $p, q > i$ jsou nejmenší indexy, na kterých se cesty opět spojují, tj. $a_p = b_q$ a vnitřní vrcholy $\{a_{i+1}, \dots, a_{p-1}\}$ a $\{b_{i+1}, \dots, b_{q-1}\}$ jsou disjunktní.

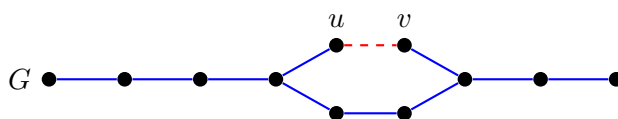
Pak ale posloupnost vrcholů $a_i, a_{i+1}, \dots, a_p = b_q, b_{q-1}, \dots, b_{i+1}, a_i$ tvoří v G kružnici, což je spor s acykličností stromu G



Dokážeme $2 \Rightarrow 1$: Z předpokladu, že mezi každými dvěma vrcholy existuje právě jedna cesta, přímo plyne souvislost G .

Pokud by G obsahoval kružnici C , pak by mezi nějakými x, y na této kružnici existovaly dvě různé cesty (po obou stranách kružnice), což je spor s acykličností stromu. Tedy G je acyklický, a proto je to strom.

Dokážeme $1 \Rightarrow 3$: Jelikož je G strom, je souvislý. Nechť $e = \{u, v\} \in E$ je libovolná hrana. Pro spor předpokládejme, že $G - e$ je stále souvislý. Pak v $G - e$ existuje nějaká u - v -cesta P . Přidáním hrany $e = \{u, v\}$ k cestě P získáme v grafu G kružnici, což je spor s tím, že G je strom.



Dokážeme $3 \Rightarrow 1$: Souvislost G je platí ze zadání. Chceme ukázat, že G je acyklický. Pro spor předpokládejme, že G obsahuje kružnici C . Zvolme libovolnou hranu $e = \{x, y\}$ na této kružnici. Odebráním hrany e se graf G nerozpadne na více komponent, protože pro jakoukoli cestu mezi vrcholy u a v , která používala hranu e , poskytne zbytek kružnice $C \setminus \{e\}$ náhradní obchůzku. Graf $G - e$ tedy zůstane souvislý, což je spor s předpokladem, že odebráním hrany se stane nesouvislým.

Dokážeme $1 \Rightarrow 4$: Souvislost G platí z definice stromu. Vztah $|E| = |V| - 1$ dokážeme indukcí podle $n = |V|$:

- *Indukční základ ($n = 1$):* Graf s jedním vrcholem nemá žádné hrany, platí $0 = 1 - 1$.
- *Indukční krok ($n - 1 \Rightarrow n$):* Dle Věty o existenci listů obsahuje strom G list v . Odebráním v vznikne graf $G' = G - v$, který je dle Věty o trhání listů opět stromem a má $n - 1$ vrcholů. Na ten můžeme použít indukční předpoklad dle, kterého platí $|E(G')| = |V(G')| - 1 = (n - 1) - 1 = n - 2$. Vrácením listu v a jeho jediné hrany zpět do grafu dostaneme $|E(G)| = |E(G')| + 1 = (n - 2) + 1 = n - 1 = |V(G)| - 1$. Tvrzení tedy platí.

Dokážeme $4 \Rightarrow 1$: Předpokládejme, že G je souvislý a $|E| = |V| - 1$. Indukcí podle $n = |V|$ dokážeme, že G je strom.

- *Základní krok ($n = 1$):* Dosadíme $|E| = 1 - 1 = 0$ tedy máme jeden vrchol a žádnou hranu, což je jistě strom.
- Pro n vrcholů nejprve ukážeme, že G obsahuje list. Z principu sudosti platí $\sum_{u \in V} \deg_G(u) = 2|E| = 2(n - 1) = 2n - 2$ (zde využíváme předpokladu $|E| = |V| - 1$). Kdyby měl každý vrchol stupeň alespoň 2, byl by součet stupňů alespoň $2n$, což nelze jelikož vychází na $2n - 2$. Tedy existuje vrchol v se stupněm nejvýše 1. Protože G je souvislý a $n > 1$, nemůže mít stupeň 0, má tedy stupeň právě 1 a je to list.
- *Indukční krok ($n - 1 \Rightarrow n$):* Odebereme-li list v , vznikne graf $G' = G - v$, odebrali jsme mu jeden vrchol a jednu hranu. Platí $|E(G')| = |E(G)| - 1 = |V(G)| - 1 - 1 = |V(G')| - 1$ a dle indukčního předpokladu je G' strom. Z věty o trhání listů je poté i $G = G' + v$ strom.

□

3.3 Kostra grafu

Nyní si zavedeme kostru grafu. To je nejmenší souvislý podgraf, který používá všechny vrcholy. Později se budeme bavit o minimální kostře. Která nám pomáhá řešit problémy, jak shrabat co nejméně sněhu, tak aby celé město propojené. Místo města, můžeme řešit třeba komunikaci v síti nebo logistický problém.

Definice 3.8 (Kostra grafu). Nechť $G = (V, E)$ je souvislý neorientovaný graf. Podgraf K grafu G nazveme **kostrou** grafu G , pokud $V(K) = V$ a K je strom.



Obrázek 1.2: Ukázka souvislého grafu G a jedné z jeho možných koster K .

Je dobré si uvědomit, že kostra grafu nemusí existovat. Pro libovolný nesouvislý graf jeho kostru nenajdete.

Pozorování 3.9 (O existenci kostry). Mějme $G = (V, E)$, jeho kostra existuje právě tehdy když je G souvislý.

Důkaz. Pro spor předpokládejme, že graf G není souvislý a má kostru K .

- Kostra je z definice strom, tedy musí být souvislá a acyklická. Dále musí procházet všechny vrcholy grafu G .
- To je ale spor s definicí, jelikož G je nesouvislý a kostra K je souvislá na všech jeho vrcholech. Tedy se nemůže jednat o podgraf.

□

Jak takovou kostru nalezneme? Vzpomeňme na algoritmus BFS, který jak víme, tvoří pole předchůdců. Každý vrchol má právě jednoho předchůdce, mezi nimi je hrana, přes kterou jsme se do tohoto vrcholu z předchůdce dostali. Pokud vezmeme z grafu všechny vrcholy a ty hrany které vedou z vrcholu do jejich předchůdce získáme tím hledanou kostru.

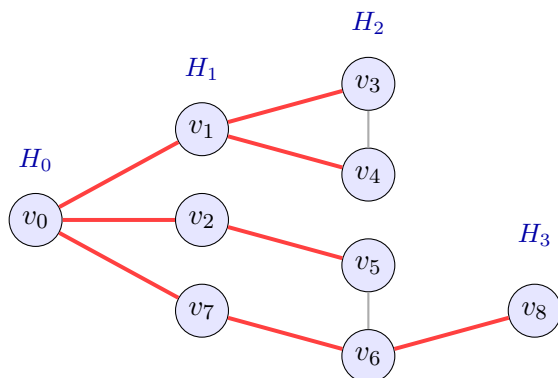
Věta 3.10 (Konstrukce kostry pomocí BFS). Nechť $G = (V, E)$ je souvislý graf a $s \in V$. Spustíme-li na G algoritmus $BFS(G, s)$, potom hrany v poli předchůdců tvoří nějakou kostru grafu G .

Důkaz. Označme $H = (V, P)$ podgraf grafu G , kde P jsou hrany pole předchůdců.

1. Všimněme si, že každý z $|V| - 1$ vrcholů (mimo kořen s) má právě jednoho předchůdce. Tedy $|E(H)| = |V| - 1$.
2. Postupným přecházením na předchůdce $v \rightarrow P[v] \rightarrow P[P[v]] \rightarrow \dots \rightarrow s$ existuje z každého vrcholu v sled do startu s . Složením u - s -sledu a s - v -sledu získáme pro libovolné $u, v \in V$ sled mezi u a v . Graf H je tedy souvislý.

Jelikož je graf H souvislý a má právě $|V| - 1$ hran, je podle Věty o charakterizaci stromů (4. ekvivalentní definice) stromem. A protože obsahuje všechny vrcholy V , je kostrou grafu G . □

Poznámka 3.11. V předešlé části textu jsme se věnovali podrobně algoritmu BFS a jeho rozdělení hladiny grafu, ač jsme vám to předtím neřekli, graf těchto hladin je kostrou grafu.



Obrázek 19: Rozklad grafu na hladiny H_0, H_1, H_2, H_3 podle vzdálenosti od v_0 . Červeně jsou zvýrazněny hrany kostry.

3.4 Topologické uspořádání

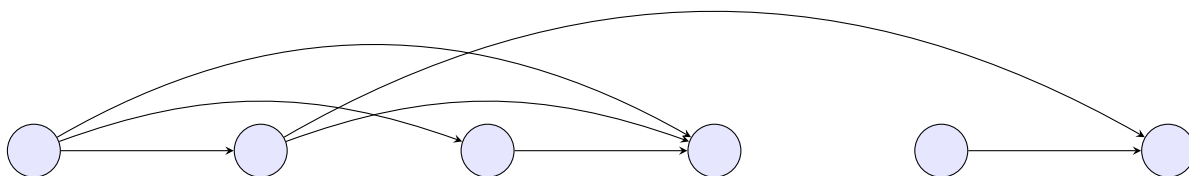
V této části se budeme věnovat orientovaným grafům.

Představte si klasický problém každého FITáka před začátkem semestru. Chce si zapsat předměty, ale ty mají různé prerekvizity. Jak tedy zjistit co si zapsat může a co ne? Přesně k tomuto problému se hodí topologické uspořádání. Podobných problémů existuje celá řada.

Může se stát, že takový problém vůbec nepůjde vyřešit? Také bychom mohli například chtít zapsat si více než jen jeden předmět, jaké si tedy může student zapsat naráz. Na všechny tyto otázky nám pomůže odpovědět následující algoritmu. Topsort.

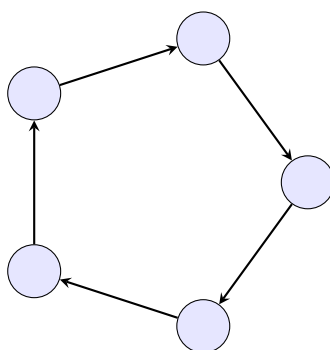
Definice 3.12 (Topologické uspořádání). Nechť $G = (V, E)$ je orientovaný graf. **Topologické uspořádání** grafu G je takové pořadí (seřazení) jeho vrcholů $v_1, v_2, \dots, v_n$, že pro každou orientovanou hranu $(v_i, v_j) \in E$ platí $i < j$.

Intuitivně, pokud nakreslíme vrcholy horizontálně do řady v pořadí topologického uspořádání zleva doprava, všechny orientované hrany (šipky) směřují výhradně zleva doprava. Tedy neexistuje šipka zpět doleva, jinak by jsme měli cyklus.



Jedno z možných topologických uspořádání

Definice 3.13. Nechť $n \geq 1$. Orientovaná kružnice délky n (s n vrcholy) je orientovaný graf $G = (\{1, \dots, n\}, \{(i, i+1) \mid i \in \{1, \dots, n-1\}\} \cup \{(n, 1)\})$.



Orientovaná kružnice

Definice 3.14 (Orientovaný acyklický graf). Orientovaný graf G nazveme **acyklickým**, pokud neobsahuje žádnou orientovanou kružnici.

Poznámka 3.15. Těmto grafům se často říká (i v české terminologii) DAG, neboli Directed Acyclic Graph.

Pokud byste takové topologické uspořádání měli vyrobit, začli byste asi tím, že byste se snažili najít vrchol, do kterého nevede žádná hrana. Takovému vrcholu budeme říkat zdroj.

Definice 3.16 (Zdroj a stok). Necht' $G = (V, E)$ je orientovaný graf.

- Vrchol $v \in V$ nazveme **zdrojem**, pokud do něj nevede žádná orientovaná hrana.
- Vrchol $v \in V$ nazveme **stokem**, pokud z něj nevede žádná orientovaná hrana.

Ale musí takový vrchol existovat? Jak ho algoritmicky najít?

Věta 3.17 (O existenci zdroje a stoku). *Necht' $G = (V, E)$ je orientovaný acyklický graf. Potom G obsahuje alespoň jeden zdroj a alespoň jeden stok.*

Důkaz. Dokážeme sporem pro zdroj (pro stok je důkaz analogický).

- Předpokládejme, že v DAGu G neexistuje žádný zdroj.
- Zvolme libovolný vrchol $v_1 \in V$. Jelikož v_1 není zdroj, vede do něj alespoň jedna orientovaná hrana z nějakého vrcholu v_2 . Jelikož ani v_2 není zdroj, vede do něj hrana z v_3 , atd.
- Tímto způsobem vytváříme posloupnost vrcholů $v_1, v_2, v_3, \dots$. Protože množina vrcholů V je konečná ($|V| = n$), musí se nejpozději po $n + 1$ krocích dle holubníkového principu nějaký vrchol zopakovat (necht' $v_i = v_j$ pro $i < j$).
- Posloupnost $v_i, v_{i+1}, \dots, v_j = v_i$ však tvoří v G orientovaný cyklus, což je spor s tím, že G je acyklický. Tedy v G musí existovat alespoň jeden zdroj.

□

3.4.1 Algoritmus TopSort

Existence zdroje nám dává cestu pro sestrojení algoritmu. V každém kroku nalezneme zdroj, zpracujeme ho a odebereme z orientovaného grafu G . Takto vyrobíme topologické uspořádání grafu G . Pokud bychom v nějakém kroku nebyli schopni nalézt zdroj grafu, pak díky větě o existenci zdroje, musí nutně obsahovat cyklus a topologické uspořádání nelze nalézt.

Jak funguje topsort: Spočítáme si vstupní stupně pro každý vrchol v (tedy počet hran co vedou do vrcholu v). Do fronty dáme vrcholy se vstupním stupněm 0, ty jsou zdroji v počátečním grafu. Vždy při odebrání zdroje z s ním odebereme i jeho hrany a každému **následníkovi** snížíme stupeň o jedna. Pokud tomuto následníkovi klesne stupeň na 0, pak se stává zdrojem a my ho přidáme do fronty.

Vstup: Orientovaný graf $G = (V, E)$.

Výstup: Topologické uspořádání vrcholů G , nebo hlášení, že graf obsahuje cyklus.

TopSort(orientovaný graf G)

- (1) $Q :=$ prázdná fronta
- (2) Pro každý vrchol $v \in V(G)$: $\delta[v] := 0$ // Pole vstupních stupňů
- (3) Pro každou hranu $(u, v) \in E(G)$: $\delta[v] := \delta[v] + 1$
- (4) Vlož do Q všechny vrcholy $z \in V(G)$ splňující $\delta[z] = 0$
- (5) Dokud fronta Q není prázdná:
 - (6) Vyjmi prvek z ze začátku fronty Q
 - (7) Vypiš z // Přidej z na další pozici topologického uspořádání
 - (8) Pro každou orientovanou hranu (z, w) vedoucí z z :
 - (9) $\delta[w] := \delta[w] - 1$
 - (10) Pokud $\delta[w] = 0$: zařaď w na konec fronty Q

- (11) Pokud nebyly zpracovány všechny vrcholy grafu G :
(12) Graf G obsahuje orientovaný cyklus!

Věta 3.18 (O správnosti algoritmu TopSort). *Algoritmus $\text{TopSort}(G)$ doběhne v konečném čase a buď vygeneruje platné topologické uspořádání, nebo správně detekuje orientovaný cyklus.*

Důkaz. Vstupní stupeň $\delta[v]$ udává počet dosud neodebraných předchůdců vrcholu v . Vrchol v je vložen do fronty Q právě tehdy, když vstupní $\delta[v] = 0$, tedy až poté, co byli všichni jeho předchůdci vyjmuti a vypisováni. Proto pro každou hranu $(u, v) \in E$ bude u vypsán dříve než v , což zaručuje korektnost výstupního pořadí.

Pokud graf obsahuje orientovaný cyklus C , žádný vrchol $v \in C$ nemůže nikdy dosáhnout $\delta[v] = 0$, protože má předchůdce na cyklu. Fronta Q se vyprázdní dříve, než budou zpracovány všechny vrcholy, a algoritmus na řádku (12) správně zahlásí cyklus. $\square$

Věta 3.19 (O složitosti algoritmu TopSort). *Při reprezentaci orientovaného grafu $G = (V, E)$ seznamem následníků má algoritmus TopSort časovou i paměťovou složitost $\mathcal{O}(|V| + |E|)$.*

Důkaz. Inicializace a výpočet vstupních stupňů trvá $\Theta(|E|)$. Každý vrchol je do fronty Q vložen a vyjmut nejvýše jednou ($\mathcal{O}(|V|)$). Každá hrana je prohledána přesně jednou při odebrání svého počátečního vrcholu ($\mathcal{O}(|E|)$). Celková časová složitost je tedy $\mathcal{O}(|V| + |E|)$. Graf G , pole δ a fronta Q zabírají nejvýše $\mathcal{O}(|V| + |E|)$ paměti. $\square$

3.5 Problém řazení a kvadratické algoritmy

S řazením se již každý z vás setkal v PAX. Cílem je uspořádat si data podle nějakého klíče. To nám následně pomůže při jejich obsluze. V této kapitole si představíme kvadratické řadící algoritmy a jejich vlastnosti. Ty se kvůli jejich složitosti vyplatí používat na menší data. Zároveň rozhodně neplatí, že by nějaké řadící algoritmy byly lepší než jiné. Některé využívají více paměti, jiné mají dobrou cache hit rate. Další zase dokáží rozpoznat již seřazené data a skončit dříve.

Definice 3.20 (Problém řazení / třídění).

Vstup: Posloupnost n porovnatelných prvků $A = (a_1, a_2, \dots, a_n)$.

Výstup: Permutace $A' = (a'_1, a'_2, \dots, a'_n)$ původní posloupnosti taková, že $a'_1 \leq a'_2 \leq \dots \leq a'_n$.

Definice 3.21 (Taxonomie řadících algoritmů). Řadící algoritmy klasifikujeme podle následujících kritérií:

1. **In-place vs. Out-of-place:** Algoritmus je *in-place*, pokud vyžaduje pouze $\mathcal{O}(1)$ (resp. $\text{polylog}(n)$) dodatečné paměti kromě samotného vstupního pole.
2. **Stabilita:** Algoritmus je *stabilní*, pokud zachovává relativní pořadí prvků se stejným klíčem. Mějme posloupnost $a_1, \dots, a_n$ pro všechny prvky a_i a a_j , kde $i < j$ a $a_i = a_j$ platí, že po seřazení bude vždy prvek a_i před a_j v seřazeném poli.
3. **Datová citlivost:** Algoritmus je *datově citlivý*, pokud jeho časová složitost závisí na konkrétních hodnotách datových prvků. Např. bubbleSort skončí dříve pokud je pole již seřazené.

3.5.1 Základní kvadratické algoritmy

SelectSort: Rozdělí pole na levou seřazenou a pravou neseřazenou část. V každém kroku najde minimum v neseřazené části a prohodí ho s jejím prvním prvkem. Ten je poté seřazený a tím se nám posune hranice mezi těmito částmi. Pokračujeme dokud celé pole není seřazené. Vlastnosti:

- **Časová složitost** je $\mathcal{O}(n^2)$.
- **In-Place.** Používá pouze $\mathcal{O}(1)$ dodatečné paměti pro uložení minima a prohození prvků.
- **Nestabilní.** V případě třeba tohoto pole [2, 2, 1]. Dojde k prohození 1 za první 2. Poté se ale dvojkám vymění pořadí ve výstupním poli. Tedy [1, 2, 2].
- **Datově necitlivý.** Algoritmus se vždy pokusí nalézt minimum i když nic neprohodí, tak pokračuje dále.

InsertSort: Rozdělí pole na levou seřazenou a pravou neseřazenou část. V každém kroku vezme první prvek z neseřazené části a odzadu ho zanořuje do seřazené části, dokud mu nenajde správnou pozici. Vlastnosti:

- **Časová složitost** je $\mathcal{O}(n^2)$.
- **In-Place.** Používá pouze $\mathcal{O}(1)$ dodatečné paměti pro prohození prvků.
- **Stabilní.** Díky cestě vzadu se vždy zastaví před stejnou hodnotou a neprohodí je.
- **Datově citlivý.** Pokud je pole seřazené či částečně seřazené, tak si ušetří spoustu zanoření a prohození. V případě seřazeného pole doběhne za $\Theta(n)$.

3.5.2 Algoritmus BubbleSort

Mějme posloupnost $a_1, \dots, a_n$. BubbleSort porovnává vždy dvojice sousedních prvků a_i a a_j pokud $a_i > a_j$ prohodí je. Takto postupně největší prvky probublají nakonec. Zároveň si drží bool proměnou $zmena$, pokud nedojde za celý cyklus k prohození, pole je již seřazené a algoritmus může skončit dříve. Zde si ho dokážeme a více zanalyzujeme.

BubbleSort(pole P , délka n)

- (1) $end := n$
- (2) $zmena := 1$
- (3) **Dokud** $zmena = 1$:
- (4) $zmena := 0$
- (5) **Pro** $j := 1, \dots, end - 1$:
- (6) **Pokud** $P[j] > P[j + 1]$:
- (7) **Prohoď** $P[j]$ a $P[j + 1]$
- (8) $zmena := 1$
- (9) $end := end - 1$

Věta 3.22 (Korektnost a invariant BubbleSortu). *Algoritmus BubbleSort korektně seřadí libovolné n -prvkové pole.*

Důkaz. Důkaz provedeme indukcí následujícího invariantu:

Invariant: Po i -té iteraci hlavního cyklu se i největších prvků pole nachází v seřazeném pořadí na posledních i pozicích vpravo.

- *Základní krok* ($i = 1$): V první iteraci algoritmus prochází celé pole zleva doprava a každou změnou posouvá největší prvek doprava. Na konci 1. iterace bude největší prvek na poslední pozici.
- *Indukční krok* ($i - 1 \rightarrow i$): Předpokládejme, že po $i - 1$ iteracích je $i - 1$ největších prvků na svých finálních pozicích vpravo. V i -té iteraci prohledáváme zbývajících $n - i + 1$ prvků. Největší z těchto zbývajících prvků probublá na pozici $n - i + 1$. Tím se pravá seřazená část rozšíří na i prvků. A invariant platí.

□

Věta 3.23 (Složitost a vlastnosti BubbleSortu). *Algoritmus BubbleSort seřadí n -prvkové pole v nejhorším čase $\mathcal{O}(n^2)$ a je **in-place**, **stabilní** a **datově citlivý**.*

Důkaz. V nejhorším případě proběhne $n - 1$ iterací, přičemž i -tá iterace trvá $\Theta(n - i)$ kroků, což dává celkem $\sum_{i=1}^{n-1} (n - i) = \Theta(n^2)$. Paměťová složitost je $\mathcal{O}(1)$. Je stabilní, neboť prohazuje pouze striktně větší prvky ($P[j] > P[j + 1]$). Pro již seřazené pole skončí po první iteraci v čase $\Theta(n)$. □

4 4. Přednáška

4.1 Zakořeněné a binární stromy

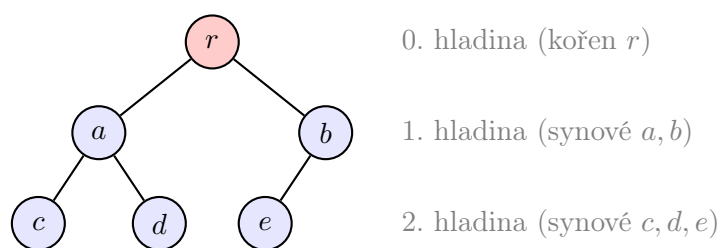
Od tohoto momentu se postupně začneme bavit o datových strukturách. Jako první si zavedeme pomocné definice zakořenění a binárního stromu. Pomocí nich budeme poté definovat další datové struktury. Např. binární vyhledávací strom nebo minimovou haldu.

Definice 4.1 (Zakořeněný strom).

- **Zakořeněný strom** je strom T , ve kterém je jeden vrchol $r \in V(T)$ označen jako **kořen**.
- Leží-li vrchol u na (jediné) cestě z vrcholu v do kořene r , pak u nazveme **předkem** vrcholu v a v nazveme **potomkem** vrcholu u .
- Pokud navíc $\{u, v\} \in E(T)$ tvoří hranu, říkáme, že u je **otec** (rodič) vrcholu v a v je **syn** (dítě) vrcholu u .
- Vrcholy rozřazujeme podle vzdálenosti od kořene do jednotlivých **hladin**: v 0. hladině leží samotný kořen r , v 1. hladině leží jeho synové atd.
- **Hloubka zakořeněného stromu** T je celkový počet jeho hladin (tedy strom o jednom prvku má hloubku 1).

Definice 4.2 (Binární strom). Strom nazveme **binární**, pokud:

1. je zakořeněný,
2. každý vrchol má nejvýše dva syny,
3. u synů rozlišujeme, který je **levý** a který **pravý**.



Obrázek 20: Ukázka zakořeněného binárního stromu s hloubkou 3.

4.2 Binární minimová halda

První datovou strukturou je binární minimová halda. Díky ní dokážeme efektivně získávat minima z množiny prvků, alternativně u maximové maxima. Takto je implementována např. prioritní fronta v c++. Dále je možné s její pomocí řadit prvky algoritmem HeapSort.

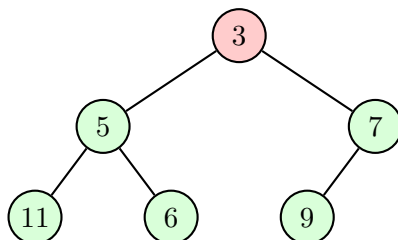
Definice 4.3 (Binární minimová halda). **Binární minimová halda** (zkráceně jen **halda**) je datová struktura ve tvaru binárního stromu, v jehož každém vrcholu x je uložen jeden porovnatelný klíč $k(x)$, přičemž splňuje dvě základní podmínky:

1. **Tvar haldy:** Strom má všechny hladiny kromě poslední zcela plně obsazené. Poslední hladina je zaplněna souvisle od levého okraje směrem k pravému.
2. **Haldové uspořádání:** Pro každý vrchol v a pro libovolného jeho syna s platí:

$$k(v) \leq k(s)$$

Poznámka 4.4. Pokud nebude řečeno jinak, binární minimovou haldu budeme vždy označovat pouze jako **halda**.

Pozorování 4.5. Z podmínek haldového uspořádání plyne, že na libovolné cestě vedoucí z jakéhokoli vrcholu nahoru do kořene tvoří klíče **nerostoucí** posloupnost. V samotném kořeni haldy se tedy vždy nachází **globální minimum** ze všech uložených prvků.



Obrázek 21: Příklad binární minimové haldy. Všimněte si, že každá rodičovská hodnota je menší nebo rovna svým synům a poslední hladina je zaplněna zleva.

Poznámka 4.6 (Atributy haldy). Pro formální práci s haldou H udržujeme následující atributy:

- $H.root$ – ukazatel na aktuální kořen haldy,
- $H.n$ – aktuální počet prvků uložených v haldě,
- $H.last$ – ukazatel na aktuálně poslední list (nejpravější prvek v nejspodnější hladině).

4.2.1 Strukturální vlastnosti haldy

Lemma 4.7 (Počet hladin haldy). *Binární halda obsahující n prvků má právě $\lfloor \log_2 n \rfloor + 1$ hladin.*

Důkaz. Uvažujme plně obsazený binární strom o $h \geq 1$ hladinách (od hladiny 0 do $h - 1$). Počet prvků je součtem geometrické řady:

$$n = 2^0 + 2^1 + 2^2 + \dots + 2^{h-1} = 2^h - 1$$

Kvůli tvaru haldy přibude do haldy nová h -tá hladina, když počet prvků n vzroste z $2^h - 1$ na 2^h . Všimněme si, že hodnota výrazu $\lfloor \log_2(n) \rfloor$ se zvětší o 1 přesně při tomto přechodu $n = 2^h$. Pro $n = 2^h - 1$ dostáváme:

$$h = (h - 1) + 1 = \left\lfloor \log_2(2^h - 1) \right\rfloor + 1 = \lfloor \log_2(n) \rfloor + 1$$

□

Kde u poslední rovnosti využijeme toho, že halda drží strukturu binárního stromu, pro $n = 2^h - 1$ (tedy úplný binární strom) má přesně $\log_2(n)$ hladin.

Lemma 4.8 (Počet listů a vnitřních vrcholů haldy). *Binární halda s $n \geq 3$ prvky má právě $\lfloor n/2 \rfloor$ vnitřních vrcholů a $\lceil n/2 \rceil$ listů.*

Tento důkaz je velmi mechanický a dokážeme ho pomocí indukce. Stačí nám zohlednit případ kdy má halda sudou a lichou velikost kvůli zaokrouhlování.

Důkaz. Důkaz provedeme matematickou indukcí podle počtu vrcholů n .

- **Základní krok** ($n = 3$): Halda má kořen (1 vnitřní vrchol) a 2 syny (2 listy). Platí $\lfloor 3/2 \rfloor = 1$ a $\lceil 3/2 \rceil = 2$. Tvrzení platí.
- **Indukční krok:** Předpokládejme, že tvrzení platí pro všechna $n' < n$. Rozlišíme dva případy podle parity n :

1. $n > 3$ **je liché:** Poslední list l je pravým synem svého otce. Jeho odebráním vznikne halda o sudé velikosti $n - 1$. Z indukčního předpokladu má tato menší halda $\lceil (n - 1)/2 \rceil$ listů a $\lfloor (n - 1)/2 \rfloor$ vnitřních vrcholů. Navrácením listu l se počet vnitřních vrcholů nezmění (jeho otec už byl vnitřním vrcholem jako otec levého syna), tedy počet vnitřních vrcholů je $\lfloor (n - 1)/2 \rfloor = \lfloor n/2 \rfloor$. Počet listů vzroste o 1:

$$\left\lceil \frac{n - 1}{2} \right\rceil + 1 = \frac{n - 1}{2} + 1 = \frac{n + 1}{2} = \left\lceil \frac{n}{2} \right\rceil$$

2. $n \geq 4$ **je sudé:** Poslední list l je jediným levým synem svého otce. Jeho odebráním vznikne halda liché velikosti $n - 1$. Navrácením listu l se jeho otec stane novým vnitřním vrcholem (přibude 1 vnitřní vrchol) a počet listů se nezmění (otec přestal být listem, ale přibyl list l). Tedy počet vnitřních vrcholů je:

$$\left\lfloor \frac{n - 1}{2} \right\rfloor + 1 = \frac{n - 2}{2} + 1 = \frac{n}{2} = \left\lfloor \frac{n}{2} \right\rfloor$$

a počet listů je $\lceil (n - 1)/2 \rceil = n/2 = \lceil n/2 \rceil$.

□

4.3 Základní operace nad haldou

Halda má tři základní operace, které provádí efektivně. A to `HeapInsert`, `HeapFindMin` a `HeapExtractMin`, které slouží pro vkládání prvků, hledání minima a smazání minima resp..

4.3.1 Vyhledání minima (`HeapFindMin`)

Nalezení minimálního prvku je triviální díky vlastnosti haldového uspořádání. Globální minimum se nachází vždy v kořeni a na ten si držíme ukazatel. To jistě stihneme za $\mathcal{O}(1)$.

`HeapFindMin`(halda H)

(1) `return  $H.root$`

4.3.2 `HeapInsert`

Při vkládání nového klíče x vytvoříme nový vrchol v a rovnou ho vložíme jako nejpravějšího syna na poslední hladině. Jenže jsme tím mohli rozbít haldové uspořádání, tedy že klíč otce o bude najednou větší než $k(v)$, $k(o) > k(v)$. V tomto případě musíme klíče $k(v)$ a $k(o)$ prohodit (druhého syna nerozbijeme, jelikož je větší než otec a toho vyměníme za menší prvek). Tento problém se může opakovat až cestou do kořene, a tak prohazujeme dokud je to potřeba. V nejextrémějším případě se z vrcholu v stane kořen, tedy globální minimum. Této operaci opravy říkáme `BubbleUp`. Díky logaritmické hloubce haldy stihneme celý `HeapInsert` za $\mathcal{O}(\log(n))$.

`HeapInsert`(halda H , klíč k)

(1) $H.n := H.n + 1$

(2) Vlož na první volnou pozici v poslední hladině nový vrchol p s klíčem k

(3) `BubbleUp`(H, p)

`BubbleUp`(halda H , uzel p)

(1) Dokud $p \neq H.root$:

(2) $o := \text{otec}(p)$

(3) Pokud $k(o) \leq k(p)$: `return`

//Halda je již opravená

(4) Prohoď $k(o)$ a $k(p)$

(5) $p := o$

Poznámka 4.9. Zde prohazujeme pouze samotné klíče, jelikož jsme si haldu definovali pouze s klíči. Pokud ale na klíči máte navázanou nějakou hodnotu nebo si někde uchováváte odkazy na vrcholy, pak je potřeba prohodit celé vrcholy, aby jste neporušili vazbu (klíč, data).

Lemma 4.10 (Korektnost a složitost `HeapInsert`). *Volání operace `HeapInsert`(H, k) nad minimovou haldou H vytvoří korektní minimovou haldu v čase $\mathcal{O}(\log(n))$.*

Důkaz. Pokud H je prázdná, pak je výsledkem jednoprvková halda. V opačném případě označme K výsledek volání `HeapInsert`(H, K). K jistě splňuje tvar haldy. Zbývá ověřit, zda splňuje podmínku haldového uspořádání.

Po přidání hodnoty k do nového listu l může být porušená podmínka mezi l a jeho otcem o , tedy $k(l) < k(o)$. Zároveň nemůžeme porušit podmínku s druhým synem r vrcholu o , jelikož pro něj platí $k(r) \geq k(o) > k(l)$. Toto řeší procedura `BubbleUp`, která prohozením posune problém o hladinu blíže ke kořeni a zastaví se nejpozději právě v něm. Již víme, že halda má $\lfloor \log_2(n) \rfloor + 1$ hladin. Na každé hladině provedeme konstantní množství operací a proto opravíme haldu za $\mathcal{O}(\log(n))$. $\square$

4.3.3 Odebrání minima (HeapExtractMin)

Kořen nemůžeme odstranit přímo, jelikož by jsme těžko hledali další minimum (museli by jsme projít celou haldu, tedy by nám operace trvala lineární čas). Proto se využívá trik. Prohodíme klíč v kořeni s klíčem $H.last$. Poslední list poté jednoduše odstraníme. Jenže náš nový klíč v kořeni nyní pravděpodobně porušuje haldové uspořádání. A tak ho postupně zabubláme dolů pomocí prohození vždy s menším z klíčů synů. Každé prohození zabere konstantní množství operací a hladin je v haldě logaritmicky, tedy to stihneme za $\mathcal{O}(\log(n))$.

HeapExtractMin(halda H)

- (1) $r := H.root, x := k(r), l := H.last$
- (2) Prohoď klíče $k(r)$ a $k(l)$
- (3) $H.n := H.n - 1$ // Odstraní poslední list l
- (4) **BubbleDown**(r)
- (5) Vrať x

BubbleDown(uzel v)

- (1) Dokud má v nějaké syny:
- (2) $s := \text{syn vrcholu } v \text{ s nejmenším klíčem}$
- (3) Pokud $k(v) \leq k(s)$: **return**
- (4) Prohoď klíče $k(v)$ a $k(s)$
- (5) $v := s$

Lemma 4.11 (Korektnost a složitost HeapExtractMin). *Volání $\text{HeapExtractMin}(H)$ odstraní a vrátí minimální prvek z haldy a zachová strukturu minimové haldy v čase $\mathcal{O}(\log(n))$.*

Důkaz. Prohozením klíčů a odstraněním posledního listu l jistě zachováme tvar haldy. Náš nový klíč v kořeni může ale porušovat haldové uspořádání. Toto opravuje procedura **BubbleDown**, která porovná vrchol v s jeho menším synem s . Pokud $k(v) \leq k(s)$ končíme. V opačném případě je prohodí. Zároveň zůstane v pořádku druhý syn d vrcholu v jelikož $k(d) \geq k(s)$, což vyžaduje haldové uspořádání. Tímto jsme problém posunuli o hladinu níže. V nejhorším případě takto pokračujeme až do listu, kde algoritmus definitivně skončí. Díky tomu, že halda má logaritmicky hladin a každou operaci provedeme v konstantním čase. Zvládneme celý **HeapExtractMin** za $\mathcal{O}(\log(n))$. □

4.4 Reprezentace haldy v poli

Jednoznačný tvar haldy nám umožňuje její reprezentaci v poli. Tato vlastnost se nám následně bude hodit pro řazení in-place pomocí algoritmu HeapSort.

Haldu postupně indexujeme zhora-dolů a zleva-doprava. Kořen začínáme indexovat na indexu 1 (snadnou úpravou se dá docílit i indexování od 0).

Poznámka 4.12 (Pozorování: Indexové vztahy v haldě). Necht' vrchol v má v poli index $i \in \{1, \dots, n\}$. Pak platí:

- Levý syn vrcholu v má index $2i$.
- Pravý syn vrcholu v má index $2i + 1$.
- Otec vrcholu v má index $\lfloor i/2 \rfloor$.
- Výraz $i \bmod 2$ udává, zda je v levým (0) nebo pravým (1) synem svého otce.

1	2	3	4	5	6
3	5	7	11	6	9

Obrázek 22: Reprezentace haldy v poli $H[1 \dots 6]$. Například uzel na indexu $i = 2$ s hodnotou 5 má levého syna na indexu $2 \cdot 2 = 4$ (hodnota 11) a pravého syna na indexu $2 \cdot 2 + 1 = 5$ (hodnota 6).

4.5 Rychlá konstrukce haldy (HeapBuild)

Na vstupu máme pole prvků a chceme z něj postavit haldu. Jednoduchý způsob je n krát zavolat operaci HeapInsert. Tímto způsobem to stihneme za $\mathcal{O}(n \log(n))$. Existuje, ale lepší způsob. Uvědomíme si všechny listy, kterých je právě $\lceil n/2 \rceil$ již splňují haldové uspořádání. Opravit musíme tedy pouze vnitřní vrcholy postupným zabubláváním dolů, od těch nejhluběji směrem ke kořeni, čímž rozšíříme opravené haldy. Většina vrcholů je v haldě "poměrně" hluboko (kvůli tvaru binárních stromů, většina prvků je na posledních hladinách), a proto nebude zabublávat přes všechny hladiny haldy. Kvalitnější analýzou zjistíme, že celá operace HeapBuild lze stihnout za $\mathcal{O}(n)$.

HeapBuild(pole $x_1, \dots, x_n$)

- (1) Vlož prvky do pole H tak, že $H[i] := x_i$ pro $i = 1, \dots, n$
- (2) Pro $i := \lfloor n/2 \rfloor$ dolů do 1:
- (3) **BubbleDown**(H, i)

Věta 4.13 (Složitost algoritmu HeapBuild). Operace *HeapBuild* postaví haldu z n prvků v časové složitosti $\mathcal{O}(n)$.

Důkaz. Uvažujme haldu o h hladinách (indexovaných $0, 1, \dots, h-1$). Na j -té hladině leží nejvýše 2^j vrcholů (pokud je plná, výjimkou může být pouze poslední hladina). Volání operace BubbleDown na prvcích na j -té hladině trvá v nejhorším případě počet hladin od j -té do nejhlubší

hladiny, tedy $\mathcal{O}(h - 1 - j)$ (poslední hladina je ve hloubce $h - 1$ a jdeme z hladiny hloubky j , tedy počet hladin mezi nimi). Sečtením práce přes všechny hladiny dostáváme celkovou složitost:

$$T(n) = \sum_{j=0}^{h-1} 2^j \cdot (h - 1 - j)$$

Pro zjednodušení substitucí $q = h - 1 - j$ přejde suma na tvar:

$$T(n) = \sum_{j=0}^{h-1} 2^{h-1-(h-1-j)} \cdot (h - 1 - j) = \sum_{q=0}^{h-1} 2^{h-1-q} \cdot q = 2^{h-1} \sum_{q=0}^{h-1} 2^{-q} \cdot q \leq n \sum_{q=0}^{h-1} \frac{q}{2^q}$$

1. *První úprava je přičtení a odečtení $h-1$ v exponentu.*
2. *Druhá úprava je substituce výrazu $h - 1 - j$ za q . Důvodem je dostat $h - 1$ do exponentu, odtud ho lze snadno vytáhnout ven.*
3. *Třetí úprava je vyjmutí výrazu 2^{h-1} ven před sumu (na sumě nijak nezávisí).*
4. *Čtvrtá úprava je kompaktní přepsání na zlomek $2^{-q} = \frac{1}{2^q}$ a zároveň platný odhad: Kde Počet prvků na poslední hladině je méně či rovno počtu prvků celé haldy.*

Časovou složitost lze zhora odhadnout nekonečnou řadou $n \sum_{q=0}^{\infty} \frac{q}{2^q}$. Jelikož výraz $a_q = \frac{q}{2^q}$ od $q \geq 1$ je kladný (první člen a_0 nemá na konvergenci vliv). Můžeme pro něj použít D’Alambertovo kritérium (viz. MA2).

$$\lim_{q \rightarrow \infty} \frac{a_{q+1}}{a_q} = \lim_{q \rightarrow \infty} \frac{q+1}{2^{q+1}} \cdot \frac{2^q}{q} = \lim_{q \rightarrow \infty} \frac{q+1}{2q} = \frac{1}{2} < 1$$

Nekonečná řada $\sum_{q=0}^{\infty} \frac{q}{2^q}$ absolutně konverguje k nějaké reálné konstantě c . Celková časová složitost je tedy $\mathcal{O}(n \sum_{q=0}^{\infty} \frac{q}{2^q}) = \mathcal{O}(c \cdot n) = \mathcal{O}(n)$. $\square$

4.6 Řadicí algoritmus HeapSort

Pomocí haldy můžeme vytvořit řadicí algoritmus HeapSort. Funguje tak, že vezmeme pole velikosti n a operací HeapBuild z něj vytvoříme haldy. Následně opakovaným voláním HeapExtractMin vytáhneme popořadě všech n prvků. Celková složitost je $\mathcal{O}(n) + \mathcal{O}(n \log(n)) = \mathcal{O}(n \log(n))$.

Důkaz korektnosti algoritmu HeapSort. Mějme pole velikosti n . Operace HeapBuild nám postaví haldy. Následně prvky odebíráme od nejmenšího. Pro spor předpokládejme, že jsem získali špatný prvek z haldy. V tom případě nám HeapExtractMin musel vrátit jiný prvek než nejmenší, ale to se nemůže stát (viz. důkaz HeapExtractMin). Tedy spor. $\square$

4.7 Amortizovaná časová složitost

Často narazíme na algoritmy, které mají "většinou" relativně malou složitost, ale jednou za čas se stane, že nějaká operace zabere opravdu mnoho času. Například zdvojnásobení velikosti pole. Znamená to tedy že složitost operace PushBack je $\mathcal{O}(n)$? V nejhorším případě samozřejmě ano, ale tak trochu tušíme, že tato informace může být poněkud zavádějící, pokud se tato drahá operace stane velmi zřídka. Můžeme se tedy podívat na dlouhodobější horizont volání této funkce nad nějakou datovou strukturou a podívat se, jaká byla v "průměru" její složitost.

Abychom se vyhlí neurčitým výrazům jako: "většinou", "průměrná operace", apod., pojďme si přesněji zadefinovat co to tedy at amortizovaná složitost je.

Definice 4.14 (Amortizovaná časová složitost). Operace A nad dynamickou datovou strukturou má v daném kontextu svého provádění amortizovanou časovou složitost $\mathcal{O}(f(n))$ (značíme $\mathcal{O}^*(f(n))$), pokud posloupnost k operací A má celkovou časovou složitost $\mathcal{O}(k \cdot f(n))$. Parametr n je velikost dynamické množiny po provedení této posloupnosti operací.

4.7.1 Příklad 1: Nafukovací pole

Uvažujme dynamické pole P s počáteční kapacitou $m = 1$. Jakmile celé pole naplníme, tak kapacitu zdvojnásobíme. Následně všechny prvky překopírujeme do nového pole s velikostí $2m$. (V reálné implementaci je šance, že se při zdvojnásobení pole stále vejde na původní místo v paměti a poté není potřeba kopírovat. My řešíme nejhorší možný případ, proto tuto možnost zanedbáváme)

NPInsert(pole P , prvek x)

(1) Nechť i je pořadové číslo vkládaného prvku.

(2) Pokud $i \leq m$:

(3) $P[i] := x$; $i := i + 1$; **return**

(4) Pokud $i > m$: // Došla kapacita pole

(5) Alokuje nové pole P' o velikosti $2m$

(6) Překopíruje prvky z P do první poloviny P'

(7) Dealokuje P ; $P := P'$; $m := 2m$;

(8) $P[i] := x$; $i := i + 1$;

Věta 4.15 (Amortizovaná složitost vkládání do nafukovacího pole). *Uvažujme na počátku prázdné nafukovací pole. Potom celková časová složitost posloupnosti n operací je $\Theta(n)$, neboli amortizovaná časová složitost jedné operace NPInsert je $\Theta^*(1)$.*

Důkaz. Práce sestává z vkládání jednotlivých prvků každý za $\mathcal{O}(1)$. Celkový čas vkládání je tedy $\Theta(n)$. Samostatné zvětšení pole zabere čas $\Theta(i)$ a to vždy když i je mocnina dvou (neboli pole je plné). Všechna zvětšení tedy zaberou:

$$2^0 + 2^1 + 2^2 + 2^3 + \dots + 2^k = \sum_{j=0}^k 2^j = \frac{2^{k+1} - 1}{2 - 1} = 2^{k+1} - 1$$

Třetí rovnost je pouze sečtení geometické řady.

Kde $2^k < n$ a zároveň 2^k je nejvyšší mocnina dvojky menší než n . Jistě platí, že $2^{k+1} - 1 \leq 2n$. (Jelikož $2^k < n$ a 2^{k+1} je 2 krát větší), tedy složitost kopírování je $\mathcal{O}(n)$. Celková časová složitost posloupnosti n operací je tedy $\Theta(n) + \mathcal{O}(n) = \Theta(n)$. Podělením počtem operací n dostáváme amortizovanou složitost operace $\Theta^*(1)$. $\square$

4.7.2 Příklad 2: Binární sčítačka

Uvažujme k -bitovou sériovou sčítačku. Operace Inc zvýší její hodnotu o jedna postupným otáčením bitů zprava do leva. V nejhorší případě musíme otočit všechny bity za $\mathcal{O}(k)$. Např. Inc na 011 otočí všechny bity na 100.

Pro dokázání počtu bitových inverzí na jednu operaci Inc (a tedy i složitost této operace) použijeme tzv. bankéřovu metodu. Ta využívá tzv. penízků. Každé operaci dáme k dispozici k penízků (např. 10, $\log(n)$, apod.), pak ukážeme, že pro každou operaci, všechny v tomto případě inverze bitů, dokážeme zaplatit z penízků co jsme dostali, popřípadě ještě z penízků co nám zbyli z minulých operací.

Věta 4.16 (Amortizovaná složitost binární sčítačky). *Uvažujme na počátku vynulovanou binární sčítačku. Celková složitost posloupnosti n volání operace Inc měřená počtem bitových inverzí je $\Theta(n)$, neboli amortizovaná složitost operace Inc je $\Theta^*(1)$.*

Důkaz. Důkaz provedeme účetní (bankéřovou) metodou.

1. Každé operaci Inc přiřadíme virtuální kredit ve výši 2 penízků.
2. Provedení jedné bitové inverze stojí právě 1 penízek.
3. První volání Inc zaplatí jeden penízek za invertování 0 na 1 a zbylí penízek dá nad tuto jedničku kterou právě invertoval.
4. Další volání Inc dokud před sebou vidí bit s hodnotou 1, tak ho invertuje na 0 do té doby, než je před ním 1. Jakmile narazí na první 0, tak ji invertuje na 1 a nechá na ní ležet penízek. Např. 1011 $\rightarrow$ 1100.
5. Protože jsme pokaždé co jsme invertovali 0 na 1 nechali na ní ležet penízek, tak každé invertování zpět na 0 můžeme zaplatit, díky tomuto našetřenému penízku.
6. Na každou operaci Inc nám stačí 2 penízky. Jeden použijeme a druhý tam necháme do budoucna. Tedy každý Inc je $\Theta^*(1)$ a složitost všech n operací je tedy $\Theta^*(1) * n = \Theta^*(n)$.

□

5 5. Přednáška

5.1 Binární vyhledávací strom

Definice 5.1. Strom nazveme **binární**, pokud

- je zakořeněný
- každý vrchol má nejvýše dva syny
- u synů rozlišujeme, který je levý a který pravý

Poznámka 5.2. Zakořenění je důležitá vlastnost. Představte si, že dostanete binární vyhledávací strom a prvek co se má do něj vložit. Kde začnete? Bez zakořenění neexistuje vstupní bod.

Definice 5.3 (Značení). Pro vrchol v binárního stromu T značíme

- $\ell(v)$ a $r(v)$: levý a pravý syn vrcholu v ,
- $L(v)$ a $R(v)$: levý a pravý podstrom vrcholu v ,
- $p(v)$: otec vrcholu v ,
- $T(v)$: podstrom tvořený kořenem v a všemi jeho potomky,
- $h(T(v))$: hloubka stromu $T(v)$ je počet hladin $T(v)$,
- $|T|$, $|T(v)|$, $|L(v)|$ a $|R(v)|$: počet vrcholů stromu T , $T(v)$, $L(v)$ a $R(v)$.

Pokud vrchol v nemá levého syna, položíme $\ell(v) = \emptyset$. Podobně pro $r(v)$ a $p(v)$. Pak dodefinujeme, že $T(\emptyset)$ je prázdný strom a $h(T(\emptyset)) = 0$.

Nyní si zavedeme binární vyhledávací strom. Kdykoliv se budeme bavit o binárních stromech je myšlen tento.

Definice 5.4. Binární vyhledávací strom (BVS) je binární strom, v jehož každém vrcholu v je uložen **unikátní** klíč $k(v)$ a pro každý jeho vrchol platí:

- Pokud $a \in L(v)$, pak $k(a) < k(v)$
- Pokud $b \in R(v)$, pak $k(b) > k(v)$

5.2 BVS Algoritmy

Poznámka 5.5. Na BVS je pár důležitých operací. Doporučuju pochopit princip těchto algoritmů než se učit pseudokód nazpaměť.

5.2.1 BVSShow

První je algoritmus BVSShow. Jeho cílem je provést InOrder průchod stromu a vypsát všechny klíče vzestupně. Z definice BVS víme, že pro každý vrchol v platí: klíče v jeho levém podstromě jsou menší a klíče v pravém podstromě větší. Aby algoritmus vypsál klíče ve vzestupném pořadí, musí se nejprve co nejvíce zanořit doleva, k nejmenším klíčům. Teprve po návratu z levého syna vypíše klíč aktuálního vrcholu a poté pokračuje do pravého syna, kde jsou klíče větší. Tento postup se opakuje pro každý vrchol stromu. Jelikož v každém vrcholu strávíme konstantní čas, celková časová složitost algoritmu je $\mathcal{O}(|T(v)|)$.

Vstup: Ukazatel na kořen v libovolného BVS $T(v)$

Výstup: Vzestupně uspořádaná posloupnost klíčů všech vrcholů v $T(v)$

BVSShow(ukazatel na vrchol v)

- (1) Pokud $v = \emptyset$: **return** // $T(v)$ je prázdný
- (2) Zavoláme **BVSShow**($\ell(v)$)
- (3) Vypíšeme $k(v)$
- (4) Zavoláme **BVSShow**($r(v)$)

5.2.2 BVSMIn

BVSMIn/BVSMax. Jeho cílem je vypsát minimální/maximální hodnotu klíče ve stromě. Pro minimální klíč se z daného vrcholu zanořujeme stále doleva. Jakmile vrchol již levého syna nemá, pak je minimálním. Analogicky pro maximum, pouze se zanořujeme doprava. V každém vrcholu provedeme konstantě operací tedy celkem provedeme $\mathcal{O}(h(T(v)))$

Vstup: Ukazatel na vrchol v libovolného BVS $T(v)$

Výstup: Minimální klíč daného BVS $T(v)$

BVSMIn(ukazatel na vrchol v)

- (1) Pokud $v = \emptyset$: **return** // $T(v)$ je prázdný
- (2) Pokud $\ell(v) = \emptyset$: **return** $k(v)$
- (3) Zavoláme **BVSMIn**($\ell(v)$)

5.2.3 BVSTFind

BVSTFind slouží k nalezení vrcholu ke klíči v BVS. Dostaneme na vstupu ukazatel na kořen v BVS a hledaný klíč. Nyní postupuje následovně, pokud je hledaný klíč větší než $k(v)$. Tak musíme jít do pravého podstromu $R(v)$, kde jsou klíče větší. Analogicky pro menší klíč. Dokud buďto nenalezneme vrchol s hledaným klíčem nebo nevylezeme ze stromu. V každém vrcholu provedeme pouze konstantě mnoho operací a celkem tedy $\mathcal{O}(h(T(v)))$.

Vstup: Ukazatel na vrchol v libovolného BVS $T(v)$ a hledaný klíč x .

Výstup: Ukazatel na vrchol s klíčem x , pokud takový v $T(v)$ existuje, jinak $\emptyset$.

BVSTFind(ukazatel na vrchol v a hledaný klíč x)

- (1) Pokud $v = \emptyset$: **return** $\emptyset$

- (2) Pokud $k(v) > x$: return **BVSFind**($\ell(v)$)
- (3) Pokud $k(v) < x$: return **BVSFind**($r(v)$)
- (4) return v

5.2.4 BVSPred

BVSPred a BVSSucc slouží k získání předchůdce/následníka nějakého vrcholu v v BVS T . Pro předchůdce, je to vždy takový klíč x , který je největší ze všech klíčů menších než x v T . Jak ho nalezneme? Rozlišujeme zde dva případy:

- Ten lehčí je když vrchol v má levého syna. V tom případě je předchůdce vrchol s maximálním klíčem z celého levého podstromu $L(v)$. To platí, jelikož jsou všechny klíče v $L(v)$ menší než $k(v)$ a maximum z $L(v)$ je ten největší menší.
- Druhý případ je když v nemá levého syna v tom případě se musíme vynořit nahoru (doprava jít nemůžeme tam jsou všechny klíče větší). Takto cestujeme směrem ke kořeni až do doby než se vrátíme do rodiče p zprava. Předchůdce je poté vrchol p . Zprava se chceme vrátit jelikož my potřebujeme najít první menší klíč. Pokud by jsme přišli zleva, tak je rodič větší než celý náš podstrom.

Pokud náš prvek je minimálním v celém stromě vrátíme nic. Alternativně lze dodefinovat vrcholy s klíči $\pm\infty$.

Analogicky platí i pro následovníka. Stačí prohodit slova levý a pravý. buďto využijeme BVSMAX nebo procházíme po rodičích a případně do kořene. celkem tedy za $\mathcal{O}(h(T(v)))$.

Vstup: Ukazatel na kořen v libovolného BVS $T(v)$ a nějaký jeho vrchol w .

Výstup: Ukazatel na předchůdce vrcholu w v BVS T . existuje-li, jinak $\emptyset$.

BVSPred(ukazatel na kořen v a vrchol w)

- (1) Pokud $l(w) \neq \emptyset$
- (2) return **BVSMAX**($l(w)$) //Máme levého syna, tedy přechůdce je maximem $L(w)$
- (3) $z := p(w)$
- (4) Dokud ($z \neq \emptyset$ & $w = l(z)$) //Jdeme směrem ke kořeni a čekáme až přijdeme zprava
- (5) $w := z$
- (6) $z := p(z)$
- (7) return z

5.2.5 BVSPred

BVSPred jak z názvu plyne vkládá prvek do BVS. Na vstupu dostaneme ukazatel na kořen v nějakého BVS a klíč x , který chceme vložit. Ve stromě předpokládáme unikátní klíče v případě již existujícího stejného klíče se vložení neprovede. Algoritmus prochází strom stejně jako BVSPred a hledá volné místo kam vložit prvek. Jakmile se dostaneme mimo strom, tak jsme našli místo kam vložit daný klíč. Pokud je strom prázdný, tak pouze vložíme prvek. V každém vrcholu provedeme pouze konstantě mnoho operací a jdeme skrze vrcholy od kořene až do listu, což je celkem za $\mathcal{O}(h(T(v)))$.

Vstup: Ukazatel na kořen v libovolného BVS $T(v)$ a vkládaný klíč x .

Výstup: Ukazatel na kořen BVS s klíčem x .

BVSPred(Ukazatel na kořen v BVS a vkládaný klíč x)

- (1) Pokud $v = \emptyset$: //Strom je prázdný

```

(2)      Vytvoř nový vrchol  $v$ 
(3)      return  $v$  in overleaf in overleaf
(4) Pokud  $k(v) > x$ :
(5)       $l(v) := \text{BVSSInsert}(l(v), x)$ 
(6) Pokud  $k(v) < x$ :
(7)       $r(v) := \text{BVSSInsert}(r(v), x)$ 
(8) return  $v$ 

```

5.2.6 BVSSDelete

Poslední operací je BVSSDelete, ten slouží k odstranění vrcholu v z BVS dle zadaného klíče. Zde máme 3 možnosti:

- První je pokud vrchol v nemá žádného syna neboli je listem. V tom případě ho prostě odstraníme a vše funguje dále.
- Další případ je kdy má právě jednoho syna vrchol w . Ani zde není problém, rodiči vrcholu v , $p(v)$ nahradíme syna v za vrchol w . Vrchol v poté bezproblému odstraníme.
- Posledním případem je kdy vrchol v má oba syny. V tomto případě nemůžeme vrchol v odstranit jinak by jsme rozbili strukturu. Pomůžeme si trikem a to pomocí nahrazení vrcholu v za jeho následníka nebo předchůdce (ti musí existovat, jelikož má vrchol v oba syny a zároveň to jsou maxima/minima těch podstromů). Také jsme nemohli rozbít definici BVS, jelikož jsme nahradili bezprostředním následníkem/předchůdcem a stále jsou vlevo menší klíče a vpravo větší.

Hledáme vrchol jako při BVSSFind, k tomu hledáme minimum podstromu pomocí BVSSMin a v každém vrcholu provedeme konstantě mnoho operací tedy celkem $\mathcal{O}(h(T(v)))$.

Vstup: Ukazatel na kořen libovolného BVS $T(v)$ a klíč x .

Výstup: Ukazatel na kořen BVS, ze kterého byl odstraněn vrchol s klíčem x , existoval-li.

BVSSDelete(Ukazatel na kořen v BVS a mazaný klíč x)

```

(1) Pokud  $v = \emptyset$ : return  $\emptyset$ 
(2) Pokud  $k(v) > x$ :
(3)       $l(v) := \text{BVSSDelete}(l(v))$ 
(4) Pokud  $k(v) < x$ :
(5)       $r(v) := \text{BVSSDelete}(r(v))$ 
(6) Pokud  $k(v) = x$ :
(7)      Pokud  $l(v) = r(v) = \emptyset$ : return  $\emptyset$  //Vrchol  $v$  je listem, mazání je triviální
(8)      Pokud  $l(v) = \emptyset$ : return  $r(v)$  //Vrchol  $v$  má pouze jednoho syna, mazání je triviální
(9)      Pokud  $r(v) = \emptyset$ : return  $l(v)$ 
(10)      $w := \text{BVSSMin}(r(v))$ 
(11)      $k(V) := k(w)$ 
(12)      $r(v) := \text{BVSSDelete}(r(v), k(w))$  //Musíme smazat ještě ten prohozený vrchol
(13) return  $v$ 

```

5.3 Časová složitost operací nad BVS

Poznámka 5.6. BVS nemá jednoznačnou strukturu. Skutečně zkuste provést BVSInsert na prvcích $\{2, 1, 3\}$ a poté na $\{1, 2, 3\}$. Množiny jsou v obou případech stejné. Ale výsledný BVS je jiný.

Časová složitost a problém BVS. Již jsme určili že operace jako BVSInsert, BVSDelate, BVSMIn, BVSTFind, atd. mají časovou složitost $\mathcal{O}(h(T(v)))$. Zde ale nastává problém, pokud nám strom zdegraduje na spojový seznam. To se stane např. pokud všechny prvky budou mít pouze pravého syna. Poté je $\mathcal{O}(h(T(v))) = \mathcal{O}(|T(v)|) = \mathcal{O}(n)$. Což se nám nelíbí, jelikož ztrácíme výhody oproti běžnému poli. Jistě lze namítnout, že šance na tento scénář je malá. V případě kdy, ale přijde útočník a začne nám dávat taková data, která toto způsobí. tak jsme nahraní. Na ochranu proti takovému zdegenerování se zavádí pojmy vyváženosti.

5.4 Dokonale Vyvážený BVS

Definice 5.7. BVS nazveme **dokonale vyvážený**, pokud pro každý jeho vrchol v platí:

$$||L(v)| - |R(v)|| \leq 1$$

Poznámka 5.8. Dokonale vyvážený BVS o velikosti n má $1 + \lfloor \log n \rfloor$ hladin. Operace na takovém stromě mají tedy optimální časovou složitost $\mathcal{O}(\log n)$.

Dokonalé vyvážení nám krásně zajišťuje logaritmickou hloubku BVS. Bohužel se ukazuje, že udržet tuto strukturu stromu současně s přidáváním a mazáním prvků, je příliš nákladné. Alespoň jedna z operací BVSInsert a BVSDelate zdegradují na lineární složitost $\mathcal{O}(n)$.

Věta 5.9 (O složitosti operací nad dokonale vyváženým BVS). *Pokud má BVS zůstat po provedení operací BVSInsert a BVSDelate dokonale vyvážený, potom ať použijeme jakoukoli implementaci zmíněných operací, bude časová složitost aspoň jedné z nich $\Omega(n)$ pro nekonečně mnoho různých n .*

Myšlenka důkazu: Stačí nám ukázat že existuje dokonale vyvážený BVS a nějaká posloupnost operací BVSInsert a BVSDelate kde alespoň jedna bude mít složitost $\Omega(n)$.

Důkaz. Použijeme důkaz přímý

1. Uvažujme dokonale vyvážený BVS s klíči $1, 2, \dots, n$, kde $n = 2^k - 1$.
2. Všimneme si, že celý tento strom je určen jednoznačně a všechna lichá čísla jsou v listech tohoto stromu.
3. Nyní provedeme dvojici operací BVSInsert($n + 1$) a BVSDelate(1), pak BVSInsert($n + 2$) a BVSDelate(2), ...
4. Po provedení i -té dvojice operací bude strom obsahovat hodnoty $i + 1, \dots, i + n$. V závislosti na tom jestli je i liché nebo sudé se vždy listech budou nacházet sudé nebo liché hodnoty. To ale znamená, že všem vrcholům se změní zda jsou listem nebo nejsou. Proto musíme upravit alespoň $\Omega(n)$ ukazatelů. Tedy alespoň jedna z operací BVSInsert nebo BVSDelate trvá $\Omega(n)$.

□

5.5 Avl strom

U úplně vyváženého BVS jsme kvůli přílišnému omezení narazili na degradaci rychlostí operací BVSInsert a BVSDelate. Proto musíme zvolit jinou cestu a to pomocí hloubkového vyvážení.

Definice 5.10. BVS je **hloubkově vyvážený**, pokud pro každý jeho vrchol v platí

$$|h(L(v)) - h(R(v))| \leq 1$$

Řešení jak toto provést je více (např. Červeno-Černé stromy viz. BI-DAS). Zde v AG1 si představíme AVL stromy. Prvně ale musíme ukázat, že opravdu toto řešení dosáhne logaritmické hloubky stromu, jinak by nemělo smysl používat.

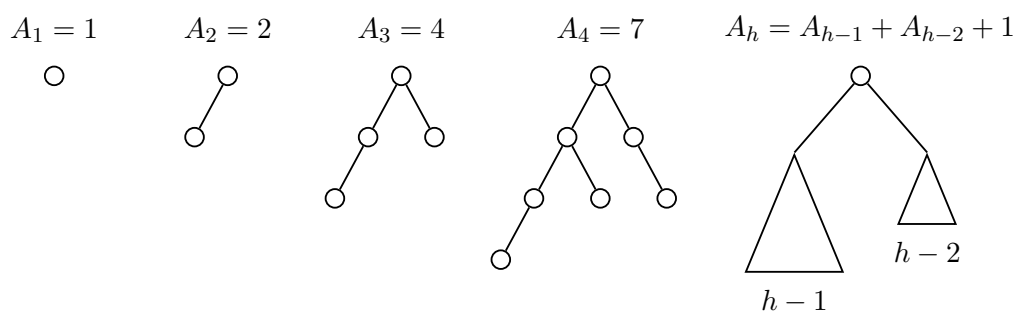
Věta 5.11 (o hloubce hloubkově vyvážených stromů). *Hloubkově vyvážený strom s n vrcholy má hloubku $\Theta(\log(n))$.*

Myšlenka důkazu: ukážeme že nejmenší možný strom (dle počtů vrcholů) stále zachovává logaritmickou hloubku což nám dá $\Omega(n)$. Poté díky tomu, že je binární víme, že pro největší možný strom platí $\mathcal{O}(n)$. Tedy celkem $\Theta(n)$.

5.5.1 Důkaz hloubky AVL

Důkaz. Důkaz provedeme indukcí.

1. Označíme si A_h jako minimální počet vrcholů hloubkově vyváženého stromu hloubky h
2. Ukážeme že počet vrcholů roste exponencionálně s hloubkou. Tedy hloubka roste logaritmicky díky inverzi těchto funkcí
3. Pro $h = 1$ je strom s jedním vrcholem. Pro $h = 2$ je to strom s 2 vrcholy ...



Obrázek 23: Rekurentní konstrukce minimálního AVL stromu $A_h = A_{h-1} + A_{h-2} + 1$.

4. Obecně platí vztah $A_h = A_{h-1} + A_{h-2} + 1$
5. Nyní dokážeme indukcí, že $A_{h+1} \geq 2^{h/2}$
6. Základní krok $A_1 = 1 \geq 2^{0/2} = 1$ a $A_2 = 2 \geq 2^{1/2} = \sqrt{2}$

7. Indukční krok:

$$\begin{aligned} A_{h+1} &= A_h + A_{h-1} + 1 \\ &> A_h + A_{h-1} \\ &= 2^{(h-1)/2} + 2^{(h-2)/2} \\ &= 2^{h/2} \cdot (2^{-1/2} + 2^{-1}) \\ &> 2^{h/2} \cdot \left(\frac{1}{2} + \frac{1}{2}\right) \\ &= 2^{h/2} \end{aligned}$$

8. Hlubkově vyvážený strom hloubky $h+1$ má tedy nejméně $2^{h/2}$ vrcholů. Proto HV strom o n vrcholech má hloubku nejvýše $\log_{\sqrt{2}}(n) + 1$

9. Protože je binární, tak nemůže mít hloubku menší než $\lfloor \log_2(n) \rfloor$

10. Tedy hloubka HV stromu o n vrcholech je $\Theta(\log(n))$

□

Poznámka 5.12. Detaily k důkazu:

1. V indukčním kroku využíváme A_{h+1} místo A_h je to kvůli případu kdy $h = 0$. V případě kdyby jsme použili A_h museli by jsme explicitně vyřešit tento případ.
2. V indukčním kroku využíváme exponenciálu $2^{h/2}$ ta se využívá jelikož to hezky vyjde. Ale je možné využít i jiný exponent.
3. Argument "protože je binární, tak nemůže mít hloubku menší než $\lfloor \log_2(n) \rfloor$ ". Lze nahradit tím stejným důkazem. Pouze nyní s největším možným stromem, aby jsme ukázali, že platí $\mathcal{O}(n)$.

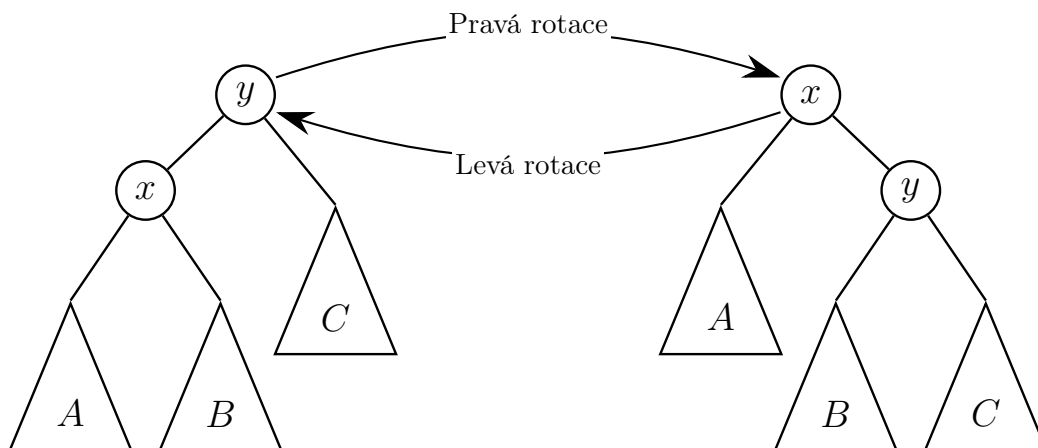
5.5.2 Rotace AVL

Definice 5.13. Znaménkem vrcholu v nazveme funkci

$$\delta(v) = h(R(v)) - h(L(v)).$$

Definice 5.14. AVL strom je hlubkově vyvážený strom, kde pro každý vrchol v platí, že $\delta(v) \in \{-1, 0, 1\}$

V případě kdy $\delta(v) \notin \{-1, 0, 1\}$, pak musíme strom opravit pomocí jednoduchých či dvojitých rotací. Obecně rotace vypadá takto:



Obrázek 24: AVL rotace: pravá rotace (y,x,A,B,C) a levá rotace.

Obrázek může prvně vypadat matoucí, ale je to kompletní návod jak je provést. Např. u pravé rotace. Vrchol y zavěšíme jako pravého syna vrcholu x . Jejich podstromy zůstanou stejné pouze podstrom B se přesune z pravého syna vrcholu x jako levý syn vrcholu y . Analogicky levá rotace.

Rotaci provádíme vždy kdy $|\delta(v)| \geq 2$ pro nějaký vrchol v . Existuje spousta možných případů kdy se jaká provádí. Nejlehčí je si napsat pár příkladů na zkoušku. Je to velmi mechanická záležitost. Pro detailní popis koukněte do přednášky.

Vstup: Vrchol x v BVS

RotateLeft(vrchol x)

- (1) $y := r(x)$
- (2) $r(x) := \ell(y)$
- (3) $\ell(y) := x$
- (4) $p(r(x)) := x$; $p(y) := p(x)$; $p(x) := y$;
- (5) $h(T(x)) := \max(h(L(x)), h(R(x))) + 1$
- (6) $h(T(y)) := \max(h(R(y)), h(T(x))) + 1$

Samotná rotace nemusí vrátit ještě AVL strom, jelikož změnou hloubky v hloubce h můžeme rozbít hloubkové vyvážení někde výše. Proto je potřeba jít vnější funkcí rekurentně až do kořene. To přesně poté dělá operace AVLInsert a AVLDelete. Nejlehčí je implementovat rotaci při návratu z rekurze.

5.5.3 Časová složitost operací nad AVL stromem

Věta 5.15. *AVLInsert a AVLDelete trvají $\mathcal{O}(\log(n))$.*

Důkaz. Důkaz provedeme přímo

1. Již víme že hloubka AVL stromu je vždy $\Theta(\log(n))$.
2. Operace na BVS byli závislé na hloubce. Tedy určitě nebudou pracovat v lepším čase než logaritmickém tedy $\Omega(\log(n))$.
3. Zároveň ale samotná rotace provede $\Theta(1)$ operací v každém vrcholu a to nejhůře na celé cestě do kořene. Což je celkem $\mathcal{O}(\log(n))$.

4. Tedy celkem $\Theta(\log(n))$.

□